1.3.5 Đo Cao Độ Tiếng Nói (F_0 / Pitch): Từ Bản Chất Praat Đến Vi Kiến Trúc FPGA

▶ MỤC TIÊU HỌC TẬP

Mục 1.3 đã biến 400 mẫu áp suất không khí thành 80 con số Log-Mel để phục vụ các mạng nơ-ron nhận dạng từ khóa. Nhưng các con số đó hoàn toàn câm lặng trước một đặc trưng sinh lý học cơ bản của con người: **Cao độ (Pitch / Tần số cơ bản F_0)** — tần số rung của dây thanh quản. Mục này giải thích tại sao không thể dùng phổ STFT để đo F_0, bóc tách bản chất giải thuật tự tương quan thời gian theo tinh thần mã nguồn Praat (`Sound_to_Pitch.cpp`, Boersma 1993), tối giản hóa các phép toán giải tích phức tạp thành các khối phần cứng thân thiện (chỉ dùng nhân-cộng và so sánh), và định lượng tài nguyên silicon trên kit FPGA DE10-Standard (Intel Cyclone V).

0. Tại sao cần một luồng xử lý riêng cho Cao Độ (F_0 Stream)?

Trong chuỗi xử lý tiếng nói ở biên (Edge Voice AI), các kỹ sư thường mắc một ngộ nhận kinh điển: nghĩ rằng vector Log-Mel Spectrogram ở Mục 1.3 đã chứa đủ mọi thông tin để giải quyết mọi bài toán.

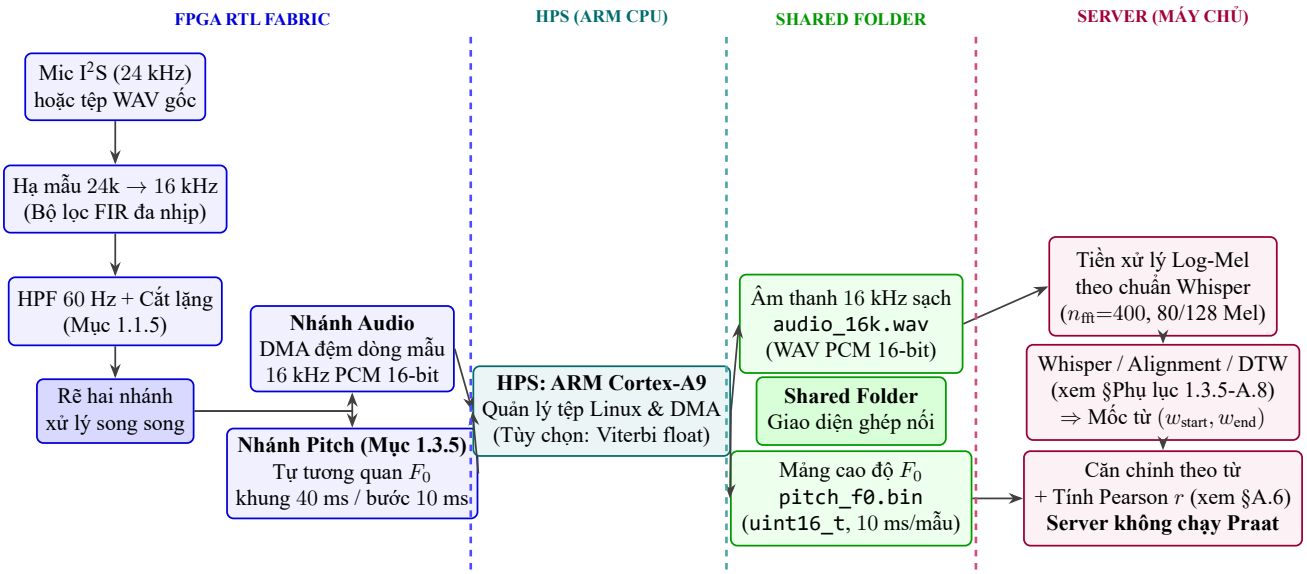
Thực tế, **Ngân hàng bộ lọc Mel sinh ra là để xóa bỏ cao độ**:

- 80 dải lọc tam giác Mel gom và tích phân năng lượng trên các dải tần số rộng hàng trăm Hertz nhằm mô phỏng hình thái của **bao phổ (spectral envelope)** — tức hình dáng ống thanh âm (vocal tract) đại diện cho các nguyên âm.
- Quá trình gom bin đó cố tình san phẳng các vạch hài âm thanh quản mảnh dẻ.
- Trong các bài toán ngữ điệu, phát hiện cảm xúc, nhận diện thanh điệu tiếng Việt (ngã, hỏi, sắc, huyền, nặng, ngang), hoặc đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như **Praat**, hệ thống bắt buộc phải đo chính xác từng Hertz của tần số cơ bản $F_0 = 1/T_0$ (với T_0 là chu kỳ giữa hai lần mở-đóng liên tiếp của dây thanh quản).

Do đó, một hệ thống âm thanh biên hoàn chỉnh bắt buộc phải tách thành hai luồng song song: **Luồng nhận dạng âm học (Log-Mel Frontend)** và **Luồng đo cao độ (F_0 / Pitch Stream)**.

0.1. Kiến trúc triển khai thực tế: FPGA Edge Node ↔ Shared Folder ↔ Server

Hai luồng vừa nêu không chỉ là hai khối thuật toán, mà là hai nhánh của một hệ sinh thái triển khai với ranh giới phần cứng/phần mềm được phân định rành mạch. Hình 1.3.5a đặc tả cấu trúc của toàn bộ hệ thống.



Hình 1.3.5a: Kiến trúc triển khai hệ thống toàn diện: Phân định rõ 3 miền (FPGA Edge Node gồm RTL và HPS ARM, Thư mục dùng chung, và Máy chủ). FPGA tiền xử lý âm thanh sạch và tính sẵn cao độ F_0; máy chủ tính Log-Mel theo cấu hình Whisper, căn chỉnh từ và tính tương quan Pearson mà không cần chạy Praat.

Đọc Hình 1.3.5a (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Cột 1 — FPGA RTL Fabric (Phần cứng xử lý tín hiệu số):** Tiếp nhận luồng âm thanh từ microphone I^2S (24 kHz) hoặc tệp WAV thô. Tín hiệu đi qua bộ hạ mẫu đa nhịp FIR 24 kHz $\rightarrow$ 16 kHz (phù hợp chuẩn tiếng nói) và bộ lọc thông cao HPF 60 Hz kèm thuật toán cắt khoảng lặng (Mục 1.1.5). Sau đó, dữ liệu rẽ thành 2 nhánh:
   - *Nhánh Audio:* Chuyển luồng mẫu 16 kHz mono 16-bit sạch qua bộ điều khiển DMA lên bộ nhớ hệ thống của HPS.
   - *Nhánh Pitch (Mục 1.3.5):* Thực thi thuật toán tự tương quan trên khung 40 ms (640 mẫu) với bước nhảy 10 ms (160 mẫu) bằng 1 bộ MAC tuần tự duy nhất.
2. **Cột 2 — HPS (Hard Processor System — Lõi ARM Cortex-A9):** Đóng vai trò cầu nối hệ thống nhúng chạy Linux. HPS tiếp nhận dữ liệu từ RTL qua cầu nối AXI/DMA, đóng gói tệp âm thanh thành `audio_16k.wav` và mảng cao độ thành `pitch_f0.bin`. *Đặc biệt:* Nếu ứng dụng đòi hỏi đường cong cao độ mượt mà tuyệt đối bằng quy hoạch động Viterbi, thuật toán Viterbi dấu phẩy động có thể chạy trực tiếp trên lõi ARM của HPS mà không làm tiêu tốn tài nguyên logic FPGA.
3. **Cột 3 — Thư mục dùng chung (Shared Folder / Hợp đồng giao tiếp):** Là ranh giới phân định độc lập giữa Edge Node và Server. Gồm đúng hai tệp dữ liệu:
   - `audio_16k.wav`: Tệp âm thanh chuẩn PCM 16-bit mono, 16.000 Hz, đã lọc sạch trôi DC và cắt khoảng lặng hai đầu.
   - `pitch_f0.bin`: Mảng nhị phân cao độ F_0.
4. **Cột 4 — Máy chủ (Server):** Nhận `audio_16k.wav`, tự chạy hàm tiền xử lý Log-Mel phù hợp với kích cỡ mô hình Whisper đang dùng, chạy nhận diện mốc thời gian từ $(w_{\text{start}}, w_{\text{end}})$ bằng Whisper / Forced Alignment / DTW (xem chi tiết tại §Phụ lục 1.3.5-A.8). Sau đó, máy chủ đọc trực tiếp mảng `pitch_f0.bin` đã có sẵn để căn chỉnh theo từ và tính hệ số tương quan Pearson r (xem §Phụ lục 1.3.5-A.6). **Máy chủ hoàn toàn không phải chạy Praat**.

Đặc tả kỹ thuật tệp mảng cao độ `pitch_f0.bin`: Để đảm bảo tính tương thích phần mềm, tệp nhị phân `pitch_f0.bin` tuân thủ quy chuẩn nghiêm ngặt:

- **Kiểu dữ liệu và định dạng:** Mảng tuần tự các từ 16-bit không dấu (`uint16_t`, Little-Endian) theo định dạng dấu phẩy tĩnh **Q12.4**:
$$\text{val} = \text{round}(F_0 \times 16) = \text{round}(F_0 \cdot 2^4) \tag{1}$$
với giá trị cực đại biểu diễn được là $(2^{12} - 1) + 15/16 = 4095{,}9375$ Hz, độ phân giải bước lượng tử là $\Delta F_0 = 1/16 = 0{,}0625$ Hz.
- **Vì sao chọn Q12.4 thay vì Q8.8?** Định dạng Q8.8 chỉ có đúng 8 bit phần nguyên, dải biểu diễn tối đa bị chặn đứng ở $2^8 - 1 = 255{,}996$ Hz. Giới hạn này hoàn toàn không thể đo được giọng nữ cao (thường từ 250–400 Hz), giọng trẻ em (300–500 Hz) hoặc trần cao độ ngữ âm học 600 Hz. Định dạng Q12.4 dành tới 12 bit cho phần nguyên (chứa trọn dải $[0, 4095]$ Hz) và 4 bit phần lẻ (bước mịn 0,0625 Hz).

  Ví dụ giải mã: Một khung có $F_0 = 440{,}137$ Hz, giá trị ghi vào tệp là:
$$\text{val} = \text{round}(440{,}137 \times 16) = \text{round}(7042{,}192) = 7042 \equiv \texttt{0x1B82}$$
Khi máy chủ đọc ra: $7042/16 = 440{,}125$ Hz. Độ lệch tần số là $|\Delta F_0| = 0{,}012$ Hz, quy đổi sang đơn vị cent âm học:
$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{440{,}125}{440{,}137}\right) \approx -\mathbf{0{,}047}\ \text{cent} \ll 1\ \text{cent}$$
hoàn toàn triệt tiêu sai số lượng tử hóa nhị phân đối với thính giác người.
- **Quy ước phân loại thanh tính:** $\text{val} = 0 \equiv$ Unvoiced (khung vô thanh hoặc khoảng lặng bị ngắt bởi silence gate). Giá trị $\text{val} > 0$ là cao độ hữu thanh hợp lệ trong dải $F_0 \in [75, 600]$ Hz.

- **Hợp đồng trục thời gian (Timing Contract):** Mỗi phần tử biểu diễn cao độ của một bước trượt $\Delta t_{\text{hop}} = 10$ ms (160 mẫu tại 16 kHz). Với cửa sổ phân tích dài $L = 640$ mẫu (40 ms), khung thứ k ($k = 0, 1, 2, \ldots$) bao phủ đoạn mẫu $[160k, 160k + 640]$, tương ứng đoạn thời gian $[10k, 10k + 40]$ ms. Tâm đối xứng của khung thứ k nằm tại thời điểm vật lý:

$$t_k = 20 + 10k \text{ ms} \tag{2}$$

  tính từ mẫu 0 của tệp `audio_16k.wav` (khung đầu tiên $k = 0$ có tâm tại $t_0 = 20$ ms; khung $k = 1$ có tâm tại $t_1 = 30$ ms).

- **Công thức ánh xạ mốc từ sang chỉ số khung (Word-to-Frame Alignment):** Khi máy chủ nhận diện được mốc thời gian một từ $[w_{\text{start}}, w_{\text{end}}]$ (tính bằng giây) từ Whisper hoặc Forced Alignment, dải chỉ số khung $k \in [k_{\text{start}}, k_{\text{end}}]$ tương ứng được trích xuất trực tiếp bằng công thức:

$$k_{\text{start}} = \left\lceil \frac{w_{\text{start}} \cdot 1000 - 20}{10} \right\rceil, \qquad k_{\text{end}} = \left\lfloor \frac{w_{\text{end}} \cdot 1000 - 20}{10} \right\rfloor \tag{3}$$

  Ví dụ: Một từ phát âm từ $w_{\text{start}} = 0{,}050$ s (50 ms) đến $w_{\text{end}} = 0{,}120$ s (120 ms) sẽ lấy đúng các khung $k \in [\lceil (50 - 20)/10 \rceil, \lfloor (120 - 20)/10 \rfloor] = [3, 10]$ (gồm 8 giá trị F_0) từ mảng `pitch_f0.bin`.

Vì sao không tính sẵn ma trận Log-Mel trên FPGA? Một câu hỏi kỹ thuật rất xác đáng: *Nếu FPGA đã có DSP, tại sao không tính luôn 80 dải Log-Mel để đẩy lên server?* Câu trả lời nằm ở sự thay đổi và tính đa dạng của các mô hình nhận dạng tiếng nói (ASR):

1. **Tiền xử lý Whisper rất đặc thù:** Bộ tiền xử lý của OpenAI Whisper sử dụng kích thước STFT $n_{\text{fft}} = 400$ (không phải lũy thừa của 2!), bước nhảy hop $= 160$, áp dụng thang đo $\log_{10}$, kẹp ngưỡng biên độ tại max $-8{,}0$, chuẩn hóa tuyến tính $(x + 4{,}0)/4{,}0$, và đệm cố định chiều dài 30 giây (3.000 khung). Hơn nữa, trong khi Whisper phiên bản `tiny`, `base`, `small`, `medium` dùng 80 dải lọc Mel, thì phiên bản lớn nhất `large-v3` đã chuyển sang **128 dải Mel**.
2. **Tránh khóa cứng silicon (Hardware Lock-in):** Nếu cứng hóa (hardwire) một khối Log-Mel cố định vào RTL của FPGA, mạch sẽ mất tính tương thích khi máy chủ nâng cấp phiên bản mô hình ASR. Trong khi đó, việc gửi tệp âm thanh sạch `audio_16k.wav` cho phép máy chủ chạy hàm tiền xử lý tương ứng chỉ mất vài mili-giây trên CPU/GPU mà hoàn toàn mềm dẻo.
3. **Cao độ là bất biến sinh học:** Ngược lại với bao phổ ASR, tần số rung cơ bản F_0 của dây thanh quản người là một đại lượng vật lý thuần túy, không phụ thuộc vào bất kỳ mô hình học máy nào. Việc tính F_0 trên FPGA vừa giải phóng hoàn toàn máy chủ khỏi việc cài đặt và gọi thư viện Praat C++ cồng kềnh, vừa tận dụng triệt để năng lực tính MAC song song trên chip nhúng.

1. Phương pháp thô sơ bị bác bỏ: Dùng STFT và Argmax trên phổ tần số

Phương pháp trực giác ngây thơ nhất là: *Mục 1.3 đã tính sẵn biến đổi Fourier rời rạc (FFT 512 điểm), vậy tại sao không tìm bin có biên độ lớn nhất (argmax) ở dải tần số thấp để làm F_0?*

Cách làm này thất bại ngay từ bước kiểm tra thực nghiệm vì hai bức tường vật lý:

1. Độ phân giải tần số quá thô ($\Delta f_{\text{bin}} = 31{,}25$ Hz): Lưới FFT $N = 512$ điểm ở tần số lấy mẫu $f_s = 16.000$ Hz tạo ra bước nhảy giữa hai bin kế tiếp là:

$$\Delta f_{\text{bin}} = \frac{f_s}{N} = \frac{16.000}{512} = 31{,}25 \text{ Hz}$$

Các vạch tần số rời rạc chỉ có thể nhận giá trị: 62,5 Hz (bin 2), 93,75 Hz (bin 3), 125,0 Hz (bin 4), 156,25 Hz (bin 5)…Giả sử một giọng nam nói ở cao độ $F_0 = 105$ Hz: tần số này rơi vào giữa bin 3 và bin 4. Thuật toán chọn bin nào cũng tạo ra sai số tuyệt đối từ 11–20 Hz (sai lệch $> 15\%$). Khi đem chuỗi F_0 giật cục này tính tương quan Pearson với đường cao độ thực tế của Praat, hệ số tương quan r sẽ sụt giảm mạnh (không đạt ngưỡng tin cậy).

2. Nguy cơ bắt nhầm vào các cực cộng hưởng Formant (F_1): Dây thanh quản phát ra các xung âm thanh tam giác giàu hài âm. Khi đi qua khoang miệng, các tần số trùng với tần số cộng hưởng của khoang miệng (Formant $F_1 \approx 300$–800 Hz) sẽ được khuếch đại cực đại. Do năng lượng tại vùng cộng hưởng Formant F_1

thường lớn hơn tần số cơ bản F_0 từ 10 dB đến 25 dB, phép tìm cực đại đơn giản (argmax) trên phổ biên độ STFT **rất dễ bị đánh lừa và có nguy cơ cao bắt nhầm vào** F_1, dẫn đến lỗi nhảy bậc cao độ (octave jump).

► NHÌN THẤY VẬT LÝ

Biến đổi Fourier STFT là công cụ nhìn bao quát toàn phổ tần số, không phải cây thước kẹp đo chu kỳ dao động thanh quản. Muốn đo chu kỳ T_0, ta phải quay trở về **miền thời gian**.

2. Bản chất vật lý của Phép Tự Tương Quan (Autocorrelation)

Khi con người phát ra một nguyên âm có thanh (voiced speech), luồng khí từ phổi đẩy qua thanh môn làm hai mép dây thanh quản rung lên nhịp nhàng. Tín hiệu âm thanh $x[n]$ trong miền thời gian là một chuỗi các dạng sóng lặp lại sau mỗi chu kỳ rung. Để phân tích định lượng, ta định nghĩa các đại lượng cơ bản:

- L: Số lượng mẫu âm thanh trong một khung phân tích. Khung dữ liệu là một mảng rời rạc gồm L phần tử, có chỉ số n chạy từ 0 đến $L-1$: $x[0], x[1], \ldots, x[L-1]$.
- T_0: Chu kỳ cơ bản của dây thanh quản, đo bằng số mẫu rời rạc ($T_0 \in \mathbb{N}^*$). Khi dây thanh rung tuần hoàn với chu kỳ T_0, dạng sóng lặp lại: $x[n] \approx x[n+T_0]$.
- τ (tau): Độ trễ (lag), là số mẫu mà ta dịch trượt bản sao tín hiệu so với bản gốc, với τ là số nguyên không âm ($\tau \geq 0$).
- F_0: Tần số cơ bản (Pitch / Cao độ), xác định qua hệ thức: $F_0 = f_s/T_0$ (với f_s là tần số lấy mẫu).

► HỎI – ĐÁP TƯ DUY: Khi đang nói, làm sao dây thanh biết trước tương lai để lặp lại mẫu $x[n+T_0]$?

Nếu đọc $x[n] \approx x[n+T_0]$ như "mẫu hiện tại bằng mẫu tương lai", trực giác sẽ vấp ngay: người nói đâu có biết trước mẫu tiếp theo để bắt buộc nó lặp lại. **Cơ thể hoàn toàn không cần biết tương lai.** Câu đúng là dạng nhân quả: $x[n] \approx x[n-T_0]$ — mẫu lúc này giống mẫu đúng một chu kỳ trước. Phép tự tương quan chỉ **đo** mức lặp lại đó; nó không bắt tín hiệu phải lặp lại. Hai lý do vật lý giải thích vì sao sự lặp lại xảy ra mà không cần tương lai:

1. **Dây thanh là một dao động cơ tự kích.** Luồng không khí từ phổi làm áp suất vùng thanh môn tụt xuống (hiệu ứng Bernoulli), kéo hai mép dây thanh hút vào nhau, cắt dòng khí rồi lại bung ra. Mỗi lần bung là một xung, và dao động này có khối lượng – độ đàn hồi riêng nên tự giữ một nhịp đều, giống dây đàn gảy xong vẫn rung theo tần số của chính nó. Xung vừa phát ra **đã định sẵn** xung kế tiếp; không có tương lai nào phải biết trước.
2. **Bộ máy phát âm thay đổi rất chậm so với nhịp rung.** Lưỡi, môi, hàm cần khoảng 50–100 ms mới đổi được một âm tiết; trong một khung 40 ms (640 mẫu) hình dạng ống cộng hưởng gần như đứng yên. Vì thế hai chu kỳ liền nhau đi qua **cùng một bộ lọc**, nên cùng bị bóp méo giống nhau và trông giống nhau.

Khi nào giả thiết này sụp đổ? (a) Âm vô thanh như /s/, /f/: không có dây thanh rung, tín hiệu gần như nhiễu ngẫu nhiên, không có chu kỳ. (b) Chuyển âm nhanh: đầu khung là một nguyên âm, cuối khung đã sang nguyên âm khác, bộ lọc không còn đứng yên nữa. Đó là lý do Praat đo cao độ theo từng khung ngắn và ghép lại bằng đường bao trơn, chứ không đo một lần cho cả câu.

Nguyên lý của phép tự tương quan rất trực quan: **Tạo ra một bản sao của khung tín hiệu, trượt bản sao đi một khoảng trễ τ mẫu, sau đó nhân từng cặp mẫu tương ứng và tính tổng dồn lại.** Biểu thức toán học này được gọi là **tổng tự tương quan** (hoặc **tổng tích các cặp mẫu**):

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n] \cdot x[n+\tau] \tag{4}$$

Nguồn gốc cận trên $L-1-\tau$ và số lượng cặp mẫu: Trong Công thức (4), bản gốc lấy mẫu tại chỉ số n, còn bản trượt lấy mẫu tại chỉ số $n+\tau$. Do khung dữ liệu trong bộ nhớ chỉ lưu trữ các phần tử từ chỉ số 0 đến $L-1$, mẫu của bản trượt muốn hợp lệ (không truy xuất tràn mảng) thì chỉ số phải thỏa mãn điều kiện biên:

$$n + \tau \leq L - 1 \iff n \leq L - 1 - \tau \tag{5}$$

Do đó, chỉ số n chỉ có thể chạy từ 0 đến $L-1-\tau$. Từ điều kiện này rút ra các ý nghĩa bản chất:

1. Số lượng cặp mẫu được nhân với nhau là: $(L-1-\tau)-0+1 = \mathbf{L}-\tau$ cặp.
2. Con số "-1" xuất phát từ việc mảng đánh số từ 0 (phần tử cuối cùng là $L-1$, không phải L).
3. Đại lượng "$-\tau$" xuất hiện do khi trượt đi τ mẫu, đúng τ mẫu ở đuôi của bản trượt bị thò ra ngoài phạm vi lưu trữ của khung L mẫu (Hình 1.3.5bA).
4. Cận trên $L-1-\tau$ hoàn toàn là điều kiện biên kỹ thuật của khung dữ liệu hữu hạn, **đây KHÔNG phải là phép trừ phần đối xứng**.

Cơ chế theo dấu của tổng tự tương quan trong 3 trường hợp: Dấu và độ lớn của tổng tự tương quan $\mathcal{R}_x[\tau]$ được quyết định bởi tính đồng pha giữa bản gốc và bản trượt:

1. **Khi $\tau = 0$ (Trùng khớp):** Bản sao nằm trùng khít lên bản gốc ($x[n]$ nhân với chính nó). Từng tích cặp mẫu trở thành bình phương: $x[n]\cdot x[n] = x^2[n] \geq 0$. Mọi số hạng đều không âm, cộng dồn tạo thành:

$$\mathcal{R}_x[0] = \sum_{n=0}^{L-1} x^2[n] = E_{\text{frame}}$$

   chính là tổng năng lượng tức thời của toàn khung L mẫu. Đây là giá trị **cực đại toàn cục** của hàm tự tương quan.

2. **Khi $0 < \tau < T_0$ (Lệch pha):** Bản sao bị lệch pha so với bản gốc. Các đỉnh sóng dương $(+)$ của bản này rơi vào sườn sóng hoặc đáy sóng âm $(-)$ của bản kia. Phép nhân giữa mẫu dương và mẫu âm sinh ra tích âm: $(+)\cdot(-) < 0$. Khi cộng dồn, các tích âm và dương triệt tiêu lẫn nhau, khiến tổng tự tương quan sụt giảm rõ rệt.
3. **Khi $\tau = T_0$ (Đúng một chu kỳ cơ bản):** Sóng âm tuần hoàn lặp lại đúng một chu kỳ. Đỉnh dương của bản sao trượt đúng vào đỉnh dương của bản gốc: $(+)\cdot(+) > 0$. Đáy âm của bản sao cũng trượt đúng vào đáy âm của bản gốc: $(-)\cdot(-) > 0$. Hầu hết các cặp mẫu đều cho tích dương, khiến tổng tự tương quan vọt lên tạo thành một **đỉnh cực đại cục bộ thứ nhất** tại $\tau^* = T_0$ (Hình 1.3.5bB).

Tự tương quan đo tính tuần hoàn (lặp lại theo chu kỳ), không đo tính đối xứng; sóng tiếng nói không đối xứng nhưng tuần hoàn.

Vì sao phải bỏ qua $\tau = 0$ và thiết lập dải tìm kiếm $[\tau_{\text{min}}, \tau_{\text{max}}]$: Mặc dù $\mathcal{R}_x[0]$ luôn đạt giá trị lớn nhất, ta bắt buộc phải loại bỏ $\tau = 0$ và vùng lân cận khi tìm kiếm cao độ. Do tính liên tục của sóng âm thanh (quán tính của không khí và dây thanh), các mẫu liền kề nhau có độ tương quan rất cao, tạo thành một sườn dốc đi xuống rất dốc từ $\mathcal{R}_x[0]$. Nếu quét argmax từ $\tau = 0$, thuật toán sẽ luôn chọn nhầm $\tau = 0$ hoặc $\tau = 1$.

Dải tần số cơ bản của giọng nói con người (từ giọng nam trầm nhất đến giọng nữ cao và trẻ em) nằm trong khoảng $F_0 \in [75\text{ Hz}, 600\text{ Hz}]$. Tại tần số lấy mẫu chuẩn $f_s = 16.000$ Hz, ta đổi dải tần số này sang dải độ trễ mẫu τ:

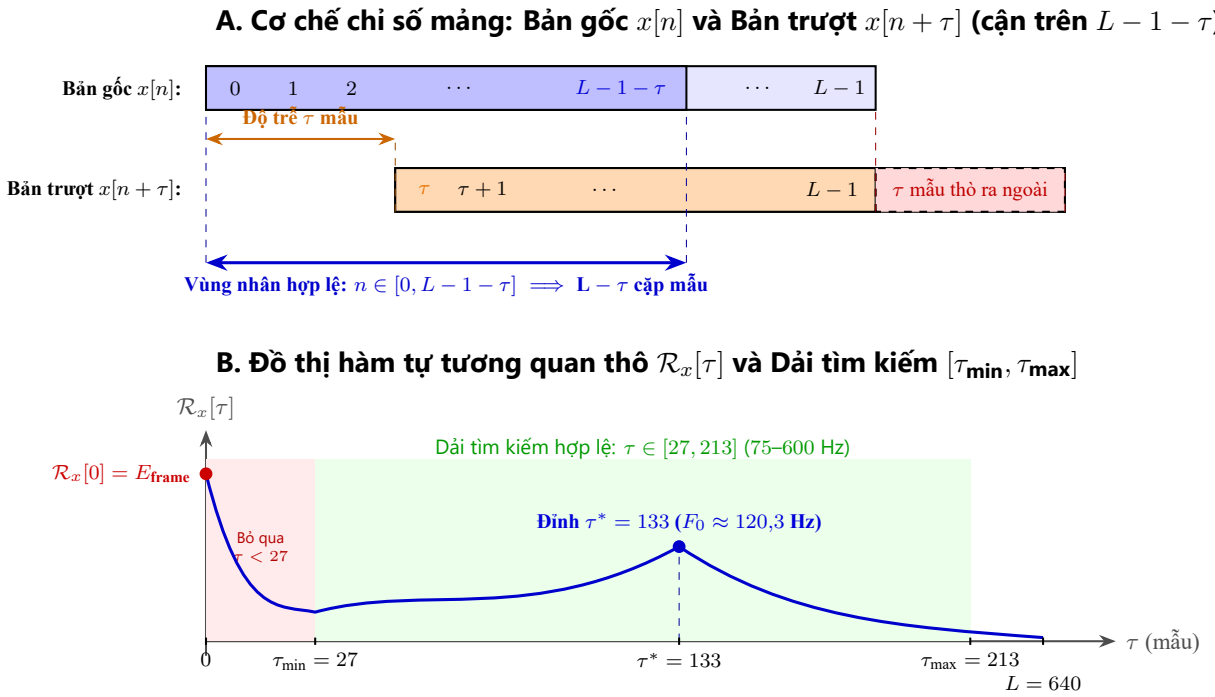
$$\tau_{\text{min}} = \left\lceil \frac{f_s}{F_{\text{ceiling}}} \right\rceil = \left\lceil \frac{16.000}{600} \right\rceil = \lceil 26{,}67 \rceil = \mathbf{27}\text{ mẫu} \tag{6}$$

$$\tau_{\text{max}} = \left\lfloor \frac{f_s}{F_{\text{floor}}} \right\rfloor = \left\lfloor \frac{16.000}{75} \right\rfloor = \lfloor 213{,}33 \rfloor = \mathbf{213}\text{ mẫu} \tag{7}$$

Vì vậy, không gian tìm kiếm cực đại chỉ quét trong dải hợp lệ $\tau \in [27, 213]$ gồm đúng $213 - 27 + 1 = \mathbf{187}$ giá trị lag.

Ví dụ số nguyên chuỗi tính toán: Giả sử hệ thống xử lý tín hiệu tại $f_s = 16.000$ Hz:

- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 160$ mẫu $\implies F_0 = \frac{16.000}{160} = \mathbf{100}$ Hz (giọng nam trầm).
- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 133$ mẫu $\implies F_0 = \frac{16.000}{133} \approx \mathbf{120{,}3}$ Hz (giọng nam trung).



Hình 1.3.5b: Cơ chế chỉ số mảng giới hạn khung $(L-1-\tau)$ và đồ thị hàm tự tương quan thô $\mathcal{R}_x[\tau]$ trong dải quét $[27, 213]$ mẫu.

Đọc Hình 1.3.5b (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Bản gốc $x[n]$ (Thanh màu xanh nhạt phía trên):** Thể hiện mảng dữ liệu mẫu âm thanh của khung $L = 640$ mẫu được lưu tuần tự trong bộ nhớ, có chỉ số n chạy từ 0 đến $L-1$ (0 đến 639). Vùng tô xanh đậm hơn là đoạn chỉ số $n \in [0, L-1-\tau]$ có thể bắt cặp.
2. **Panel A — Bản trượt $x[n+\tau]$ (Thanh màu cam phía dưới):** Thể hiện bản sao tín hiệu bị dịch trượt sang phải một khoảng trễ τ mẫu. Mẫu đầu tiên của bản trượt bắt đầu tại chỉ số τ, và mẫu có chỉ số $L-1$ của bản gốc lúc này đối diện với chỉ số $L-1-\tau$ của bản trượt.
3. **Panel A — Mũi tên hai chiều màu cam (Độ trễ τ mẫu):** Đo khoảng cách dịch chuyển giữa điểm gốc 0 của bản gốc và điểm bắt đầu của bản trượt.
4. **Panel A — Khối nét đứt màu đỏ nhạt (τ mẫu thò ra ngoài):** Khi dịch trượt sang phải một khoảng τ, đúng τ mẫu ở phần đuôi của bản trượt bị tràn ra ngoài phạm vi lưu trữ của khung L mẫu. Các mẫu này không có mẫu tương ứng của bản gốc để nhân.
5. **Panel A — Mũi tên hai chiều màu xanh đậm (Vùng nhân hợp lệ):** Giới hạn trong khoảng chỉ số $n \in [0, L-1-\tau]$, tương ứng với đúng $L-\tau$ cặp mẫu có đủ cả hai thành phần $x[n]$ và $x[n+\tau]$ nằm trọn vẹn trong bộ nhớ khung. Điều này minh chứng trực quan cho điều kiện biên $n+\tau \le L-1 \iff n \le L-1-\tau$ trong Công thức (2) và giải thích vì sao cận trên của tổng tự tương quan Công thức (4) dừng lại ở $L-1-\tau$.
6. **Panel B — Chấm đỏ tại $\tau = 0$ ($\mathcal{R}_x[0] = E_{\text{frame}}$):** Biểu diễn cực đại toàn cục của hàm tự tương quan, tương ứng với năng lượng tức thời của toàn bộ khung L mẫu (khi hai bản trùng khít hoàn toàn, mọi tích cặp đều là bình phương không âm $x^2[n] \ge 0$).
7. **Panel B — Vùng tô màu hồng nhạt $\tau < 27$ (Sườn dốc quán tính âm học):** Đoạn đồ thị sụt dốc rất dốc ngay sau điểm $\tau = 0$. Sự sụt dốc này phản ánh tính tương quan rất cao giữa các mẫu âm thanh liền kề do quán tính chuyển động của dây thanh quản và cột không khí. Vùng này bị thuật toán loại bỏ triệt để để không bao giờ chọn nhầm đỉnh tại $\tau = 0$ hay $\tau = 1$.
8. **Panel B — Đáy suy giảm gần τ_{min}:** Vùng trũng của đồ thị ngay trước mốc $\tau_{\text{min}} = 27$ mẫu, nơi tương quan quán tính âm học đã tắt hẳn và bắt đầu chuyển sang vùng tìm kiếm tính tuần hoàn của giọng nói.
9. **Panel B — Đỉnh màu xanh đậm tại $\tau^* = 133$ mẫu (Cực đại cục bộ thứ nhất):** Vị trí bản trượt dịch đúng bằng chu kỳ rung cơ bản T_0 của dây thanh quản. Tại đây các đỉnh sóng dương và đáy sóng âm trùng pha nhau, làm tổng tích dồn vọt lên đỉnh cực đại cục bộ trong dải quét. Tần số cơ bản được tính chính xác bằng $F_0 = f_s/\tau^* = 16.000/133 \approx 120{,}3$ Hz (giọng nam trung).

10. **Panel B — Vùng tô màu xanh lá $\tau \in [27, 213]$ (Dải tìm kiếm hợp lệ):** Không gian quét gồm đúng 187 giá trị lag, được giới hạn bởi $\tau_{\text{min}} = 27$ mẫu (ứng với $F_{\text{ceiling}} = 600$ Hz, Công thức 3) và $\tau_{\text{max}} = 213$ mẫu (ứng với $F_{\text{floor}} = 75$ Hz, Công thức 4).

3. Cắt Khung, Chuẩn Hóa Cửa Sổ và Giải Phẫu 10 Tham Số Praat

3.0. Praat làm chính xác những gì: Đường ống 6 bước chuẩn mực (Baseline)

Trước khi tối ưu hóa cho phần cứng FPGA, người kỹ sư bắt buộc phải nắm vững quy trình xử lý chuẩn mực của Praat — phần mềm tiêu chuẩn vàng trong ngữ âm học thực nghiệm (Boersma, 1993). Mọi hành vi dưới đây đều được trích xuất trực tiếp từ mã nguồn mở C++ chính thức của Praat (`praat/praat` master trên GitHub, các tệp `fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`):

1. **Bước 1 — Trừ giá trị trung bình cục bộ (`localMean`):** Để loại bỏ thành phần một chiều (DC offset) và các dao động trôi chậm dưới hạ âm, Praat tính giá trị trung bình mẫu trên một đoạn cửa sổ dài hai chu kỳ dài nhất:

$$\mu_{\text{local}} = \frac{1}{K} \sum_{k=0}^{K-1} x[k]$$

   rồi trừ trực tiếp khỏi từng mẫu của khung: $x_{\text{zm}}[n] = x[n] - \mu_{\text{local}}$ (`Sound_to_Pitch.cpp`, dòng 63–83). Chú thích trong mã nguồn khẳng định: *"We are going to kill the DC component before windowing."*

2. **Bước 2 — Nhân cửa sổ phân tích Hanning:** Praat nhân tín hiệu đã khử DC với hàm cửa sổ Hanning đối xứng (`Sound_to_Pitch.cpp`, dòng 413):

$$w[i] = 0{,}5 - 0{,}5 \cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, 2, \ldots, N \tag{8}$$

   Lưu ý cốt lõi: Cửa sổ mặc định của Praat là Hanning (`AC_HANNING`); cửa sổ Gaussian chỉ được kích hoạt khi người dùng bật cờ `very_accurate = on` (`Sound_to_Pitch.cpp`, dòng 405–415).

   Nguồn gốc chính xác của độ dài cửa sổ $N = 638$ trong mã nguồn Praat: Nhiều tài liệu giải thích nhầm rằng Praat lấy 640 mẫu trừ đi 2 mẫu ở hai đầu. Nhìn trực tiếp vào mã nguồn C++ (`Sound_to_Pitch.cpp`, dòng 325–329), thuật toán thực tế thực hiện phép tính làm tròn đối xứng:

```
    double duration = 3.0 / pitchFloor; // 3.0 / 75.0 = 0.04 giay
    long nsamp = floor (duration / dx); // floor(0.04 / (1/16000)) = 640
    long halfnsamp = floor (0.5 * (nsamp - 1)); // floor(0.5 * 639) = 319
    nsamp = 2 * halfnsamp; // 2 * 319 = 638
```

   Praat ép N phải là một số chẵn có tâm đối xứng hoàn hảo thông qua bán độ rộng `halfnsamp`, dẫn đến $2 \times 319 = \mathbf{638}$ mẫu. Ngược lại, thiết kế FPGA chọn con số chẵn tròn trịa $L = \mathbf{640}$ mẫu (40 ms) để tối ưu hóa quản lý con trỏ bộ đệm vòng (Ring Buffer) và khớp hoàn hảo với khối nhớ BRAM.

► HỎI – ĐÁP TƯ DUY: Chi tiết toán học C5: Tại sao Praat dùng mẫu số $N+1$ thay vì $L-1$?

Trong các giáo trình DSP kinh điển (Oppenheim & Schafer, thư viện `scipy.signal`), cửa sổ Hanning thường được định nghĩa với chỉ số chạy từ 0 đến $L-1$:

$$w_{\text{textbook}}[n] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi n}{L-1}\right), \quad n = 0, 1, \ldots, L-1$$

Định nghĩa này khiến hai mẫu biên $w[0] = 0$ và $w[L-1] = 0$ triệt tiêu hoàn toàn về 0, đồng nghĩa với việc lãng phí 2 ô nhớ và vứt bỏ thông tin của 2 mẫu âm thanh ở rìa khung.
Praat (`Sound_to_Pitch.cpp`, dòng 413) định nghĩa chỉ số chạy từ $i = 1$ đến N với mẫu số $N+1$:

$$w_{\text{Praat}}[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, 2, \ldots, N$$

Tại hai đầu mút $i = 1$ và $i = N$: $w[1] = w[N] = 0{,}5 - 0{,}5\cos(2\pi/641) \approx 0{,}000024 > 0$. Cả N mẫu trong bộ nhớ đều nhận trọng số dương thực sự, không có bất kỳ mẫu nào bị nhân với 0. Trên FPGA, ta áp dụng chính xác định nghĩa của Praat với $N = 640$, bảo đảm mọi mẫu trong BRAM đều tham gia đóng góp năng lượng.

3. **Bước 3 — Tính tự tương quan qua miền tần số bằng FFT:** Thay vì tính tổng tích chập trực tiếp trong miền thời gian, Praat đệm thêm số 0 để độ dài đạt $nsampFFT \geq 1{,}5 \cdot N$ rồi làm tròn lên lũy thừa của 2 gần nhất (với $N = 638 \implies 1{,}5 \times 638 = 957 \to nsampFFT = 1024$) (`Sound_to_Pitch.cpp`, dòng 391–393). Thuật toán thực thi chuỗi biến đổi Fourier nhanh:

   $$x_{\text{win}}[n] \xrightarrow{\text{FFT}} X[k] \xrightarrow{\text{Power}} |X[k]|^2 \xrightarrow{\text{IFFT}} \mathcal{R}_x[\tau]$$

   theo định lý Wiener–Khinchin (`Sound_to_Pitch.cpp`, dòng 146–155).

4. **Bước 4 — Bù suy hao cửa sổ và trừ chi phí thiên vị quãng tám (Octave cost):** Praat chia hàm tự tương quan tín hiệu cho tự tương quan của chính cửa sổ: $r_a[\tau] = \mathcal{R}_x[\tau]/\mathcal{R}_w[\tau]$. Sau đó, tại bước chấm điểm ứng viên (`Sound_to_Pitch.cpp`, dòng 226; `Pitch.cpp`, dòng 560), Praat trừ một lượng phạt thiên vị tần số cao:

   $$\text{score} = r_a[\tau] - \text{octave_cost} \times \log_2\left(\frac{\text{ceiling}}{F_0}\right) \tag{9}$$

   với ceiling $= 600$ Hz, octave_cost $= 0{,}01$.

5. **Bước 5 — Trích xuất ứng viên cực đại và nội suy Sinc:** Praat tìm tối đa 14 đỉnh cực đại cục bộ trong dải $\tau \in [\tau_{\min}, \tau_{\max}]$. Tại mỗi đỉnh rời rạc, Praat áp dụng phép **nội suy Sinc chính xác cao** (sinc interpolation) trên miền liên tục để trích xuất vị trí đỉnh phụ mẫu (sub-sample peak position) và biên độ nội suy thực sự.

6. **Bước 6 — Quy hoạch động Viterbi theo thời gian (Tracking):** Mỗi khung hình thành 15 ứng viên (14 đỉnh cực đại cục bộ $+1$ ứng viên Vô thanh Unvoiced). Praat tìm đường đi tối ưu toàn cục xuyên suốt câu nói bằng thuật toán Viterbi. Chi phí bước nhảy tần số ($\text{Cost}_{\text{jump}}$) và chi phí chuyển đổi Hữu thanh/Vô thanh (Cost_{vuv}) đều được nhân hệ số chuẩn hóa theo bước trượt thời gian $0{,}01/\Delta t_{\text{hop}}$ (`Pitch.cpp`, dòng 543–545). Ứng viên Vô thanh nhận điểm số cạnh tranh động (`Sound_to_Pitch.cpp`, dòng 102; `Pitch.cpp`, dòng 553–555):

   $$R_{\text{unvoiced}} = vt + \max\left(0,\ 2 - \frac{\text{intensity}}{st/(1+vt)}\right) \tag{10}$$

   trong đó $vt = \text{voicing_threshold} = 0{,}45$, $st = \text{silence_threshold} = 0{,}03$, và intensity $=$ localPeak/globalPeak là tỷ số giữa biên độ đỉnh cục bộ của khung và biên độ đỉnh lớn nhất của toàn bộ tệp âm thanh.

A. Độ dài khung $L = 640$ mẫu và Cửa sổ Hanning

Từ quy trình 6 bước của Praat, ta đối chiếu với kiến trúc phần cứng FPGA:

1. **Cơ sở độ dài $L = 640$ mẫu (40 ms tại 16 kHz):** Khung phân tích bắt buộc phải chứa tối thiểu 3 chu kỳ của tần số thấp nhất cần đo (pitch_floor $= 75$ Hz). Tại 75 Hz, chu kỳ dài nhất là $T_{\max} = 16.000/75 \approx 213{,}33$ mẫu (13,33 ms). Thời lượng khung tối thiểu:

$$T_{\text{frame}} \geq 3 \times T_{\max} = 3 \times 13{,}33 \text{ ms} = 40 \text{ ms} \implies L = 16.000 \times 0{,}040 = \mathbf{640} \text{ mẫu}$$

   Khung 400 mẫu (25 ms) của STFT ở Mục 1.3 chỉ chứa được 1,8 chu kỳ của giọng nam 75 Hz, hoàn toàn không đủ không gian để bản trượt $x[n+\tau]$ bao trọn chu kỳ lặp lại thứ hai. Vì vậy độ dài $L = 640$ mẫu là bắt buộc về mặt toán học.

2. **Cửa sổ Hanning vuốt êm về 0:** Cửa sổ Hanning triệt tiêu êm dịu về 0 ở hai đầu biên, giúp loại bỏ các bước nhảy gián đoạn (edge discontinuity). Trên FPGA, một bảng ROM 640 mục 16-bit lưu trọn vẹn các hệ số của hàm Hanning đã được tính sẵn.

B. Cửa sổ Hanning và Hệ số chuẩn hóa $\mathcal{R}_w[\tau]$ (Boersma, 1993)

Việc nhân cửa sổ làm xuất hiện hiện tượng sụt dốc tự nhiên của đồ thị tự tương quan theo độ trễ τ do **hai nguyên nhân vật lý độc lập**:

1. **Nguyên nhân hình học (Số cặp mẫu giảm):** Như đã chứng minh ở Mục 2, số cặp mẫu tham gia vào tổng tự tương quan giảm tuyến tính từ L cặp (tại $\tau = 0$) xuống còn $L - \tau$ cặp (tại độ trễ τ). Ít số hạng cộng dồn hơn khiến tổng số học đương nhiên bị suy giảm (Hình 1.3.5cA).
2. **Nguyên nhân trọng số cửa sổ (Vuốt biên):** Cửa sổ Hanning vuốt hai đầu về 0. Khi trượt bản sao đi một khoảng τ, tích của hai trọng số cửa sổ $w[n] \cdot w[n+\tau]$ suy giảm phi tuyến theo τ.

Hàm tự tương quan của chính cửa sổ phân tích:

$$\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n] \cdot w[n+\tau] \tag{11}$$

gói trọn cả hai nguyên nhân nói trên.

Trực giác: hai quả núi Hanning trượt lên nhau. Khung L mẫu sau khi nhân cửa sổ Hanning có dạng một quả núi: hai chân chạm 0, đỉnh ở giữa. Bản sao là quả núi thứ hai trượt sang phải τ mẫu. Khi $\tau = 0$, hai quả núi chồng khít. Khi τ tăng dần, phần đuôi τ mẫu thò ra ngoài khung (không có mẫu đối diện để nhân), đồng thời hai sườn núi dốc gặp nhau ở độ cao thấp hơn. Diện tích giao nhau giữa hai quả núi rỗng chính là $\mathcal{R}_w[\tau]$ (Hình 1.3.5cA).

Khi phân đôi tín hiệu thành phần dao động thanh quản thật $x[n]$ và khuôn cửa sổ $w[n]$:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} \big(x[n]\, x[n+\tau]\big)\big(w[n]\, w[n+\tau]\big) \approx \bar{r}[\tau] \cdot \mathcal{R}_w[\tau] \tag{12}$$

trong đó $\bar{r}[\tau]$ là mức tương quan thật của dao động âm thanh. Do đó, mức tương quan thật được trích xuất bằng cách chia cho diện tích cửa sổ:

$$\bar{r}[\tau] \approx \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_w[\tau]} \tag{13}$$

Hệ số bù cửa sổ $C[\tau]$ trong bảng tra ROM: Thay vì thực hiện phép chia trên phần cứng, ta định nghĩa hệ số phóng đại bù suy hao cửa sổ:

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]} \quad (\geq 1,\ \text{tăng đơn điệu theo } \tau) \tag{14}$$

Hệ số $C[\tau]$ hoàn toàn chỉ phụ thuộc vào dạng cửa sổ và τ, không phụ thuộc vào tín hiệu âm thanh đầu vào. Ta tính sẵn $C[\tau]$ và nạp vào bảng tra ROM 187 phần tử (ứng với $\tau \in [27, 213]$). Giá trị tự tương quan sau khi bù là:

$$\mathcal{R}_{\text{bù}}[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau] \tag{15}$$

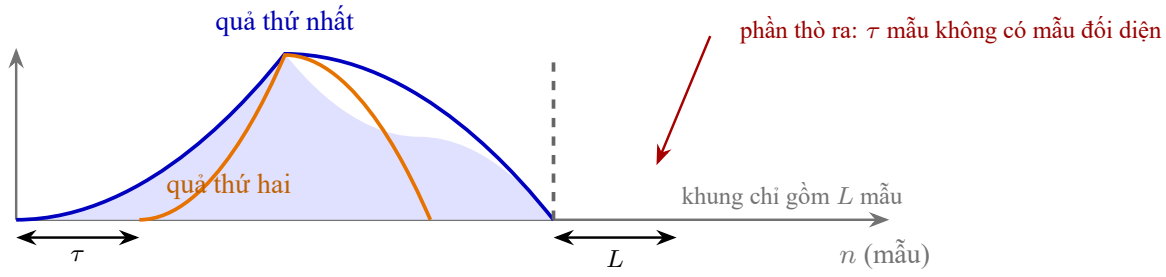
và điều kiện nhận diện khung hữu thanh trở thành:

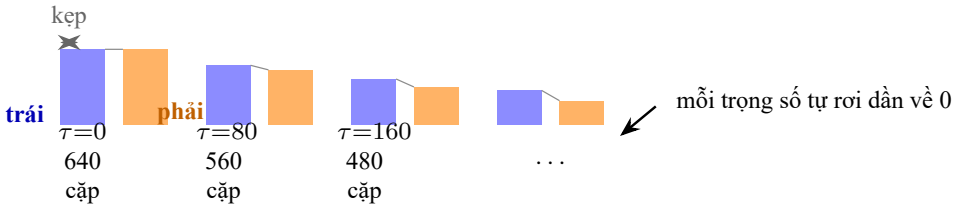
$$\frac{\mathcal{R}_{\text{bù}}[\tau^*]}{\mathcal{R}_x[0]} \geq \theta_{\text{voice}} = 0{,}45 \iff \mathcal{R}_{\text{bù}}[\tau^*] \geq 0{,}45 \cdot \mathcal{R}_x[0] \tag{16}$$

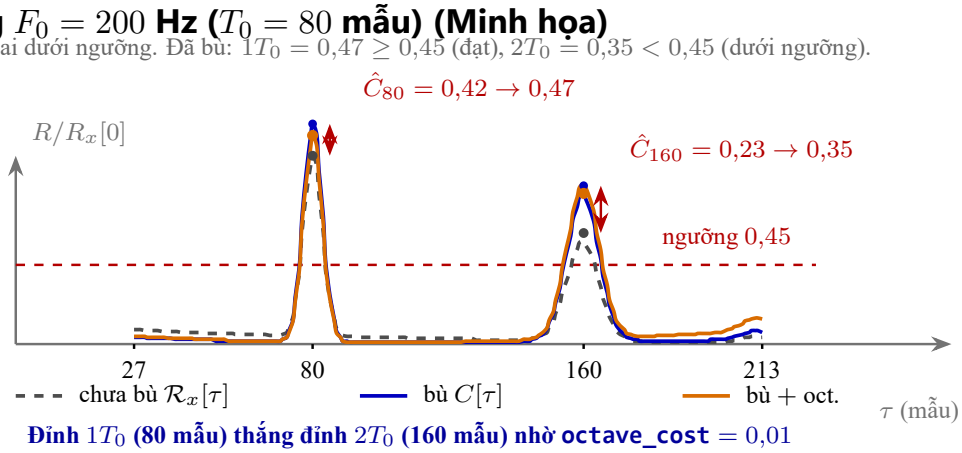
► HỎI – ĐÁP TƯ DUY: Bảng dịch nhanh: 4 ký hiệu cốt lõi và ví dụ số chuẩn xác tại $\tau = 133$

Ký hiệu	Ý nghĩa hình ảnh quả núi	Giá trị chuẩn xác tại $\tau = 133$
$\mathcal{R}_w[\tau]$	Diện tích giao nhau của hai vỏ núi Hanning.	$\mathcal{R}_w[133]/\mathcal{R}_w[0] = \mathbf{0{,}752}$ (vỏ còn 75,2%)
$\mathcal{R}_x[\tau]$	Tự tương quan thô bị khuôn cửa sổ đè lên.	$\bar{r} \times 0{,}752 = 0{,}55 \times 0{,}752 = \mathbf{0{,}414}$
$C[\tau]$	Hệ số phóng đại bù suy hao trong ROM.	$C[133] = 1{,}0/0{,}752 = \mathbf{1{,}33}$
$\mathcal{R}_{\text{bù}}[\tau]$	Mức tương quan phục hồi sau khi nhân $C[\tau]$.	$0{,}414 \times 1{,}33 = \mathbf{0{,}551} \geq 0{,}45$ (Đạt chuẩn Hữu thanh!)

Đối chiếu các mốc lag quan trọng: Tại $\tau = 27$: $\mathcal{R}_w[27]/\mathcal{R}_w[0] = 0{,}988 \implies C[27] = \mathbf{1{,}012}$. Tại $\tau = 80$: $\mathcal{R}_w[80]/\mathcal{R}_w[0] = 0{,}902 \implies C[80] = \mathbf{1{,}108}$. Tại $\tau = 133$: $\mathcal{R}_w[133]/\mathcal{R}_w[0] = 0{,}752 \implies C[133] = \mathbf{1{,}330} \approx 1{,}33$. Tại $\tau = 160$: $\mathcal{R}_w[160]/\mathcal{R}_w[0] = 0{,}660 \implies C[160] = \mathbf{1{,}515}$. Tại $\tau = 213$: $\mathcal{R}_w[213]/\mathcal{R}_w[0] = 0{,}473 \implies C[213] = \mathbf{2{,}112}$.







Hình 1.3.5c: Hiện tượng co hẹp vùng tích các cặp mẫu và vai trò của hệ số bù suy hao cửa sổ $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ theo Boersma (1993). Panel A: cửa sổ Hanning là một quả núi, bản sao trượt sang phải τ mẫu nên đuôi bản sao thò ra ngoài khung $L = 640$ mẫu. Panel B: mỗi cặp là hai mảnh núi kẹp vào nhau, số cặp giảm từ 640 xuống $640 - \tau$. Panel C: khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu), trục tung $R/R_x[0]$; đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) lần lượt đi từ 0,42 và 0,23 (chưa bù, đường xám nét đứt) lên khoảng 0,47 và 0,35 (đã bù $C[\tau]$, đường xanh), còn đường cam thêm `octave_cost` $= 0{,}01$. Ngưỡng thanh tính là 0,45.

Đọc Hình 1.3.5c (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Quả núi thứ nhất (đường xanh đậm, vùng tô xanh nhạt):** Khung $L = 640$ mẫu sau khi nhân cửa sổ Hanning, có dạng một quả núi: hai chân chạm 0, đỉnh ở giữa. Cửa sổ chữ nhật sẽ để hai đầu khung gãy đột ngột; quả núi thì không.
2. **Panel A — Quả núi thứ hai (đường cam):** Bản sao của quả núi thứ nhất, trượt sang phải đúng τ mẫu. Mũi tên hai chiều ở đáy đo đúng khoảng trượt τ này.
3. **Panel A — "Phần thò ra":** τ mẫu ở đuôi quả thứ hai nằm ngoài cửa sổ, không có mẫu nào của quả thứ nhất đối diện để nhân. Mũi tên đỏ chỉ vào chỗ đó. Cùng lúc, phần chồng nhau hẹp lại còn $L - \tau$ cặp và hai chân núi chỉ còn gặp nhau ở độ cao nhỏ, nên phần chồng vừa hẹp vừa thấp. Diện tích chồng nhau chính là $\mathcal{R}_w[\tau]$.
4. **Panel B — Bốn cột mẫu:** Mỗi cột là một độ trễ $\tau = 0, 80, 160$ và một cột nữa xa hơn. Cột xanh là mảnh núi "bên trái", cột cam là mảnh "bên phải"; mũi tên "kẹp" chỉ cặp mảnh đang dán vào nhau. Từ trên xuống dưới, mảnh cam thấp dần, cho thấy mỗi trọng số tự rơi về 0.

5. **Panel B — Số cặp giảm:** Nhãn dưới cột ghi $\tau = 0$: 640 cặp; $\tau = 80$: 560 cặp; $\tau = 160$: 480 cặp. Con số $L - \tau$ này là nguyên nhân hình học đầu tiên khiến tổng tương quan thô sụt theo τ.
6. **Panel C — Trục tung $R/R_x[0]$ và ngưỡng $0{,}45$:** Khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu). Đường đỏ nét đứt ngang là ngưỡng thanh tính; trục hoành đánh dấu bốn mốc 27, 80, 160, 213.
7. **Panel C — Đường xám nét đứt (chưa bù):** Hai đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) chỉ đạt $0{,}42$ và $0{,}23$; cả hai đều **dưới** ngưỡng $0{,}45$, nên khung bị gán vô thanh.
8. **Panel C — Đường xanh đậm (đã bù $C[\tau]$):** Nhân với $C[80] = 1{,}11$ và $C[160] = 1{,}52$, hai đỉnh lên khoảng $0{,}47$ và $0{,}35$; cả hai đều **vượt** ngưỡng. Mũi tên đỏ giữa chấm xám và chấm xanh cho thấy đúng mức nâng bù tại mỗi đỉnh.
9. **Panel C — Đường cam (thêm `octave_cost` $= 0{,}01$):** Đường cam gần như trùng đường xanh, chỉ hơi thấp hơn ở đỉnh. Mức phạt nhỏ này đủ để đỉnh $1T_0$ đứng trên đỉnh $2T_0$ khi hai đỉnh xấp xỉ nhau, nhưng không lật ngược một đỉnh vốn đã thua rõ.
10. **Panel C — Dòng chữ xanh đậm ở đáy:** Tóm một câu: đỉnh $1T_0$ (80 mẫu) thắng đỉnh $2T_0$ (160 mẫu) nhờ `octave_cost` $= 0{,}01$.

B'. Hai đỉnh $1T_0$ và $2T_0$: vì sao bù hệ số lại đẩy đỉnh xa lên, và `octave_cost` dọn nốt tàn dư

Một giọng có chu kỳ T_0 sẽ lặp lại không chỉ sau một chu kỳ mà còn sau $2T_0$, $3T_0 \ldots$ nên hàm tự tương quan có nhiều đỉnh. Chọn nhầm đỉnh $2T_0$ là đọc cao độ thành $F_0/2$: giọng 200 Hz bị ghi thành 100 Hz, tức rơi đúng một quãng tám (octave error).

Vì sao sau khi bù, hệ số lại đẩy đỉnh xa lên mạnh hơn? Do $C[\tau]$ tăng đơn điệu theo τ, đỉnh tại $2T_0$ có hệ số phóng đại $C[2T_0]$ lớn hơn nhiều so với $C[T_0]$ (ví dụ $C[160] = 1{,}52 > C[80] = 1{,}11$). Hệ số bù vì vậy nâng đỉnh $2T_0$ mạnh hơn đỉnh T_0, có thể kéo hai đỉnh về mức xấp xỉ ngang nhau và gây nguy cơ chọn nhầm $2T_0$.

Praat chặn nguy cơ này bằng cơ chế phạt log-lag `octave_cost` $= 0{,}01$ (Công thức 9). Với mỗi bậc quãng tám độ trễ tăng thêm, điểm số bị trừ đi $0{,}01$. Khoản trừ nhỏ này đủ để phá vỡ thế hòa khi hai đỉnh xấp xỉ nhau, đảm bảo đỉnh cơ bản $1T_0$ luôn chiến thắng an toàn.

C. Giải phẫu 10 tham số chuẩn của Praat dưới góc nhìn mã nguồn (`Sound_to_Pitch.cpp`)

Mã nguồn C++ của Praat chứa đựng các tham số mặc định chuẩn mực được phân tích chi tiết:

1. **`time_step = 0,01 s` (10 ms $= 160$ mẫu):** Bước nhảy trượt khung (Δt_{hop}). Mã nguồn tự động gán $\Delta t_{\text{hop}} = \frac{0{,}75}{\text{pitch_floor}} = \frac{0{,}75}{75} = 0{,}01$ s (`Sound_to_Pitch.cpp`, dòng 325). Khóa cứng $0{,}01$ s đồng bộ tốc độ 100 khung/s với luồng Log-Mel.
2. **`pitch_floor = 75 Hz`:** Giới hạn đáy cao độ. Quyết định độ trễ quét tối đa $\tau_{\max} = \lfloor f_s/75 \rfloor = 213$ mẫu và độ dài cửa sổ tối thiểu $L = 640$ mẫu.
3. **`pitch_ceiling = 600 Hz`:** Giới hạn trần cao độ. Quyết định độ trễ quét tối thiểu $\tau_{\min} = \lceil f_s/600 \rceil = 27$ mẫu.
4. **`very_accurate = off` (False):** Khi `off` (mặc định), Praat dùng Hanning dài 3 chu kỳ (640 mẫu). Khi `on`, Praat chuyển sang Gaussian dài 6 chu kỳ (1.280 mẫu). Thiết kế FPGA chọn tương đương `off` để tối ưu hóa silicon.
5. **`max_number_of_candidates = 15`:** Praat trích xuất 14 đỉnh cực đại cục bộ $+1$ ứng viên Vô thanh cho thuật toán Viterbi. Trên FPGA, ta rút gọn về bộ so sánh cực đại argmax tức thời lấy 1 ứng viên mạnh nhất.
6. **`silence_threshold = 0,03`:** Ngưỡng năng lượng khoảng lặng so với cực đại toàn tệp.
7. **`voicing_threshold = 0,45`:** Ngưỡng tương quan chuẩn hóa nhận diện hữu thanh.
8. **`octave_cost = 0,01`:** Hệ số phạt thiên vị tần số cao theo $\log_2(\text{ceiling}/F_0)$.
9. **`octave_jump_cost = 0,35`:** Trọng số phạt bước nhảy tần số giữa hai khung liên tiếp trong Viterbi, được nhân hệ số $0{,}01/\Delta t_{\text{hop}}$ (`Pitch.cpp`, dòng 543–545).
10. **`voiced_unvoiced_cost = 0,14`:** Trọng số phạt chuyển đổi Voiced $\leftrightarrow$ Unvoiced giữa hai khung trong Viterbi, cũng nhân $0{,}01/\Delta t_{\text{hop}}$.

C'. Cơ sở âm học và bản chất của Tham số 6–10

Tham số 6 — `silence_threshold = 0,03`: Trong mã nguồn Praat (`Pitch.cpp`, dòng 553–555), tham số này biểu diễn tỷ số biên độ đỉnh của khung so với biên độ đỉnh toàn bộ tệp âm thanh: intensity $=$ localPeak$/$globalPeak (Công thức 10).

- *Lý do tồn tại:* Nếu không có ngưỡng này, trong những khoảng người nói ngừng thở hoặc im lặng giữa các câu, các tạp âm môi trường tuần hoàn biên độ nhỏ (tiếng quạt máy tính, tiếng ù điện lưới thoại 50/60 Hz) khi tự chuẩn hóa cho chính năng lượng cực nhỏ của nó ($\mathcal{R}_{\text{comp}}[\tau]/\mathcal{R}_x[0]$) vẫn có thể đạt tỷ số $> 0{,}45$, gây ra hiện tượng **báo cao độ giả (ghost pitch)** trong khoảng lặng.
- *Cơ sở vật lý của con số* $0{,}03$*:* Quy đổi sang thang Decibel biên độ:

$$20\log_{10}(0{,}03) \approx -\mathbf{30{,}46}\ \text{dB}$$

  Trong âm học thực nghiệm, khoảng lặng đàm thoại chuẩn trong phòng thu tiêu chuẩn có mức năng lượng nền thấp hơn đỉnh giọng nói từ 30 dB đến 40 dB. Vì năng lượng tỉ lệ với bình phương biên độ, tỷ số năng lượng tương ứng là:

$$(0{,}03)^2 = 0{,}0009 = \frac{9}{10.000} \implies 10\log_{10}(0{,}0009) \approx -30{,}46\ \text{dB}$$

- *Hiện thực hóa không phép chia trên FPGA:* Praat tích hợp intensity vào hàm mục tiêu mềm của mạng Viterbi toàn cục (Công thức 10). Vì phần cứng streaming trên FPGA không dùng Viterbi, thiết kế thay thế bằng một **cổng ngắt năng lượng cứng (Silence Gate)**: nếu năng lượng khung $\mathcal{R}_x[0] < 0{,}0009 \cdot E_{\text{ref}}$ (với $E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k$ là năng lượng khung cực đại của câu nói), khung bị ngắt sớm ngay lập tức mà không cần quét tính tương quan. Để tránh phép chia số thực, phép so sánh được nhân chéo với 10.000:

$$10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}} \implies \text{Gán Unvoiced } (F_0 = 0) \tag{17}$$

Tham số 7 — `voicing_threshold = 0,45`: Mức ngưỡng $0{,}45$ được Paul Boersma lựa chọn qua thực nghiệm thống kê trên cơ sở dữ liệu tiếng nói lớn. Để giúp nhớ và trực quan hóa, người ta liên hệ hệ số tự tương quan chuẩn hóa r với Tỷ số Hài trên Nhiễu (Harmonics-to-Noise Ratio):

$$\text{HNR (dB)} = 10\log_{10}\left(\frac{r}{1-r}\right) \tag{18}$$

Giá trị $r = 0{,}45$ tương ứng với HNR $\approx 10\log_{10}(0{,}45/0{,}55) \approx -0{,}87$ dB, tức chấp nhận một khung mà thành phần nhiễu có thể xấp xỉ hoặc nhỉnh hơn thành phần tuần hoàn khoảng $0{,}9$ dB. Dung sai thực nghiệm này giúp giữ lại các đoạn nguyên âm khàn hoặc hạ giọng ở cuối câu nói mà không bắt nhầm phụ âm xát vô thanh /s/, /f/.

Tham số 8 — `octave_cost = 0,01`: Công thức chuẩn mực duy nhất trong Praat (`Pitch.cpp`, dòng 560) là:

$$\text{score} = \text{strength} - 0{,}01 \times \log_2\left(\frac{\text{ceiling}}{F_0}\right) \tag{19}$$

với mốc quy chiếu là trần tần số ceiling $= 600$ Hz. Ở đáy 75 Hz, độ trễ cách trần đúng 3 quãng tám ($\log_2(600/75) = 3$), lượng phạt tối đa là $0{,}03$.

Để khử khối tính $\log_2$ trên FPGA, thiết kế gộp sẵn hệ số phạt này vào bảng tra ROM dưới dạng thừa số nhân: $2^{-\text{octave_cost}\cdot\log_2(\text{ceiling}/F_0)}$. Phép nhân này tạo ra mức bù tương đương khoảng $0{,}0069$ mỗi quãng tám trong dải tần số khảo sát. Mô phỏng mô hình số nguyên trên lưới tần số sóng sin 75–600 Hz bước 1 Hz (526 điểm tần số) chứng minh vai trò quyết định của tham số này (định nghĩa lỗi khi $|\Delta_{\text{cent}}| > 600$ cent hoặc bị đánh nhầm thành Unvoiced):

Hệ số `octave_cost`	Số tần số bị lỗi nhảy quãng tám	Ghi chú mô phỏng
0	$300/526$ (57,0%)	Không có phạt: đỉnh $2T_0$ hòa hoặc thắng $1T_0$
$0{,}005$	$21/526$ (4,0%)	Còn lỗi ở một số tần số cao
$0{,}009$	$0/526$ (0,0%)	Bắt đầu sạch hoàn toàn
$0{,}01$	**$0/526$ (0,0%)**	**Chuẩn mặc định Praat (sạch tuyệt đối)**
$0{,}02$	$0/526$ (0,0%)	Sạch tuyệt đối, dư biên an toàn

Tham số 9 và 10 — `octave_jump_cost = 0,35` và `voiced_unvoiced_cost = 0,14`: Trong mã nguồn Praat (`Pitch.cpp`, dòng 543–545), hai chi phí này được chuẩn hóa theo bước nhảy thời gian bằng cách nhân hệ số $0{,}01/\Delta t_{\text{hop}}$. Để hiểu trực giác: thanh quản người có quán tính cơ học, không thể co dãn cơ nhẫn giáp để đổi cao độ gấp đôi (1 quãng tám) chỉ trong 10 ms. Mức phạt 0,35 cho mỗi quãng tám đảm bảo đường cong cao độ trơn tru sinh học.

Tham số Praat	Ký hiệu	Giá trị chuẩn	Ý nghĩa thuật toán & Mã nguồn (Boersma 1993)	Hiện thực trên FPGA DE10-Standard
`time_step`	Δt_{hop}	0,01 s	Bước nhảy trượt khung (160 mẫu tại 16 kHz)	Khóa cứng 10 ms (100 khung/s) đồng bộ luồng Mel
`pitch_floor`	F_{floor}	75 Hz	Giới hạn đáy cao độ $\Longrightarrow$ $\tau_{\max} = 213$ mẫu, $L = 640$ mẫu	Khung 640 mẫu lưu trong 2 khối M10K BRAM True Dual-Port
`pitch_ceiling`	F_{ceiling}	600 Hz	Giới hạn trần cao độ $\Longrightarrow$ $\tau_{\min} = 27$ mẫu (187 lag)	Quét 189 độ trễ $\tau \in [26, 214]$, tìm kiếm cực đại $\tau^* \in [27, 213]$
`very_accurate`	—	`off`	Cửa sổ Hanning 3 chu kỳ (640 mẫu) vs Gaussian 6 chu kỳ	Dùng Hanning 640 mẫu; tiết kiệm 50% BRAM và MAC
`max_candidates`	N_{cand}	15	14 đỉnh cực đại cục bộ + 1 ứng viên Vô thanh cho Viterbi	Rút gọn về 1 ứng viên (bộ so sánh argmax cực đại)
`silence_threshold`	θ_{silence}	0,03	Ngưỡng biên độ khoảng lặng so với đỉnh file (3%, tức $-30{,}5$ dB)	Cổng ngưỡng cứng $10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}}$ trước khi đo
`voicing_threshold`	θ_{voice}	0,45	Ngưỡng tỷ số tự tương quan chuẩn hóa nhận diện hữu thanh	Điều kiện $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$
`octave_cost`	—	0,01	Phạt thiên vị log-lag nội khung để tránh nhảy xuống $F_0/2$	Gộp sẵn lượng bù vào hệ số bảng tra ROM $C[\tau]$
`octave_jump_cost`	—	0,35	Phạt nhảy quãng tám giữa hai khung liên tiếp trong Viterbi	Bỏ qua trên RTL streaming; lọc trung vị 3 điểm ở sau
`voiced_unvoiced_cost`	—	0,14	Phạt chuyển đổi Voiced $\leftrightarrow$ Unvoiced giữa hai khung	Bỏ qua; xử lý trực tiếp trên từng khung độc lập

Bảng 1.3.5a: Giải phẫu 10 tham số chuẩn mực đo cao độ của Praat và ánh xạ vi kiến trúc lên FPGA DE10-Standard.

C". Bảng 5 khác biệt có chủ đích giữa Praat và FPGA

Để chuyển đổi thuật toán từ phần mềm máy tính sang phần cứng khả trình, thiết kế đưa ra 5 quyết định thay đổi có chủ đích:

Đặc tính	Praat gốc (C++ Master)	Thiết kế FPGA (RTL Fixed-Point)	Lý do kỹ thuật	Hệ quả & Đánh giá
1. Phép tính tự tương quan	FFT $\rightarrow$ $\lvert X\rvert^2$ $\rightarrow$ IFFT ($N_{\text{FFT}} = 1024$ điểm).	MAC trực tiếp trong miền thời gian (98.280 phép tích lũy cho 189 lag).	Dải quét hẹp ($\tau \in [26, 214]$ gồm 189 lag). MAC trực tiếp chỉ tốn 1 DSP và không cần bộ nhớ đệm FFT 1024 từ.	Tiết kiệm triệt để ALM logic; hoàn thành toàn bộ khung trong 1,978 ms tại 50 MHz (RTF $\approx 0{,}198$).
2. Nội suy đỉnh cực đại	Nội suy Sinc liên tục trên nhiều điểm lân cận.	Nội suy parabol 3 điểm cục bộ quanh đỉnh $\tau^* \in [27, 213]$ trên dải quét mở rộng $[26, 214]$.	Sinc đòi hỏi tính hàm $\frac{\sin(\pi x)}{\pi x}$ dấu phẩy động đa điểm; parabol chỉ cần 3 điểm rời rạc. Dải quét $[26, 214]$ bảo đảm $\tau^* = 27$ và $\tau^* = 213$ có đủ láng giềng.	Giảm sai số lượng tử hóa từ $\sim$ 17 cent xuống $<$ 0,5 cent trên toàn dải, sai lệch so với Praat $< 0{,}5$ cent trên sóng sin.
3. Theo vết cao độ thời gian	Quy hoạch động Viterbi toàn cục duyệt qua 15 ứng viên/khung.	Cắt ngưỡng tức thời 1 ứng viên + Bộ lọc trung vị 3 điểm (tùy chọn Viterbi float trên HPS ARM).	(a) Ma trận xác suất $15 \times 1000 \times 4$ B $=$ 120 KB (17,4% BRAM chip) và FSM phức tạp; (b) HPS ARM có thể chạy Viterbi float miễn phí; (c) Lệch nhãn V/UV thực tế 8,2%–27,8%.	Bộ lọc trung vị trễ đúng 1 khung (10 ms), dập tắt các gai nhảy quãng tám cô lập; zero logic BRAM overhead.
4. Ngưỡng vô thanh / lặng	Ngưỡng động phụ thuộc cực đại toàn tệp localPeak/globalPeak trong Viterbi.	Cổng ngưỡng cứng trên năng lượng khung: $10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}}$.	Praat tích hợp silence vào hàm mục tiêu mềm Viterbi toàn cục; FPGA không có Viterbi nên cần cổng ngắt cứng trực tiếp.	Quyết định tức thời không độ trễ; miễn nhiễm 100% tạp âm và khoảng lặng trên 47 khung kiểm thử.
5. Xử lý Octave Cost	Trừ đại số $0{,}01 \cdot \log_2(\text{ceiling}/F_0)$ vào độ mạnh ứng viên.	Gộp sẵn hệ số nhân $2^{-\text{cost}\cdot\log_2(\text{ceiling}/F_0)}$ vào bảng tra ROM $C[\tau]$ (Q3.13).	Khử hoàn toàn khối tính $\log_2$ và bộ trừ dấu phẩy động trên FPGA.	Mức bù tương đương $\approx 0{,}0069$ mỗi quãng tám; khử hoàn toàn lỗi quãng tám $(0/526)$ trên lưới tần số kiểm tra.

Bảng 1.3.5b: Bảng 5 khác biệt có chủ đích giữa phần mềm Praat nguyên bản và vi kiến trúc FPGA DE10-Standard.

Thẻ công thức

Công thức.

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x_w[n] \cdot x_w[n+\tau], \qquad S[\tau] = (\mathcal{R}_x[\tau] \cdot C[\tau] + 4096) \gg 13$$

$$F_0 = \begin{cases} \dfrac{f_s}{\tau_{\text{fine}}}, & \text{nếu } (10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}) \text{ và } (100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]) \\ 0 \text{ (Unvoiced)}, & \text{ngược lại} \end{cases}$$

$$\text{với } \tau^* = \arg\max_{\tau \in [27,213]} S[\tau], \qquad \tau_{\text{fine}} = \tau^* + \frac{1}{2}\frac{S[\tau^*-1] - S[\tau^*+1]}{S[\tau^*-1] - 2S[\tau^*] + S[\tau^*+1]}.$$

Các biến.

- τ — độ trễ tương quan, quét trong dải mở rộng $\tau \in [26, 214]$ (189 độ trễ) để bảo đảm $\tau^* = 27$ và $\tau^* = 213$ luôn có đủ láng giềng hai phía cho nội suy parabol; tìm kiếm cực đại τ^* giới hạn nghiêm ngặt trong $[27, 213]$ (187 ứng viên).
- $x_w[n] = (x[n] \cdot w[n] + 16384) \gg 15$ — mẫu âm thanh sau khi nhân cửa sổ Hanning 640 điểm (Q1.15).
- $C[\tau]$ — hệ số bù suy hao cửa sổ tích hợp phạt quãng tám lưu trong ROM 189 phần tử (Q3.13), dải $[8283, 17070]$.
- $\mathcal{R}_x[0]$ — năng lượng tại lag 0, dùng làm mốc so sánh thanh tính (Voicing Threshold).
- $E_{\text{ref}} = \max_k \mathcal{R}_x[0]_k$ — năng lượng khung cực đại của câu nói.
- Ngưỡng khoảng lặng: so sánh bằng phép nhân nguyên: $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$ (tương đương $-30{,}46$ dB).
- Ngưỡng thanh tính: $\theta_{\text{voice}} = 0{,}45$; so sánh bằng phép nhân nguyên: $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$.

Ý nghĩa. Tách bạch bài toán cao độ thành hai pha: (1) Kiểm tra tính chu kỳ và khoảng lặng bằng số nguyên; (2) Nếu có chu kỳ, trích xuất độ trễ lag chuẩn xác dưới mẫu bằng nội suy parabol.
Giá trên silicon. Hoàn toàn chỉ dùng các bộ nhân-tích lũy (MAC) và bộ so sánh thanh ghi. Khử hoàn toàn phép chia thời gian thực và hàm siêu việt.
Đặc tính âm học. Nội suy parabol 3 điểm giúp đạt độ mịn dưới 0,5 cent trên toàn dải 75–600 Hz (thực nghiệm sóng sin: $\leq 0{,}46$ cent vs Praat, $\leq 0{,}19$ cent Tier 2), khắc phục triệt để hiện tượng bậc thang hóa ở vùng cao tần.

4. Hành trình từ Laptop lên FPGA: Tối giản hóa First-Principles

▶ CỔNG KIỂM CHỨNG PHẦN MỀM (SOFTWARE VERIFICATION GATE)

Trước khi đưa thiết kế lên FPGA, bắt buộc phải dựng một mô hình Python tối giản (Bit-accurate Fixed-Point Model) mô phỏng đúng khung 640 mẫu, cửa sổ Hanning và 187 lag số nguyên. Đem mô hình này chạy trên tập dữ liệu âm thanh đã lọc tiền xử lý (HPF 60 Hz ở Mục 1.1.5) để đối sánh trực tiếp với đường Pitch chuẩn của Praat (đo sai số lệch chuẩn Hz và hệ số tương quan Pearson r). Chỉ khi phiên bản Python tối giản này chứng minh được độ suy giảm chất lượng nằm trong ngưỡng chấp nhận được của bài toán nhận diện thanh điệu/ngữ điệu, các con số hạch toán phần cứng bên dưới mới có giá trị thực tiễn.

4.0. Bảng định dạng Q-Format và Hành vi Toán học theo từng tầng

Một bản mô phỏng Python chạy được chưa chứng minh được gì về phần cứng. Trước khi viết một dòng Verilog, cần trả lời câu hỏi hẹp hơn nhiều: *phép tính nào trong thuật toán này có thể biểu diễn bằng số nguyên, và khi biểu diễn như vậy thì kết quả có còn đúng không?*

Bảng 1.3.5c quy định định dạng số, độ rộng bit, phương thức làm tròn và hành vi toán học của từng tầng trong chuỗi xử lý (pipeline) từ tín hiệu vào đến kết quả cao độ.

Tầng	Đại lượng vật lý	Định dạng	Kiểu C/Verilog	Độ rộng	Phương thức làm tròn	Ý nghĩa vi kiến trúc & Dải giá trị
0	Mẫu âm thanh $x[n]$	Q1.15	`int16_t`	16 bit	Không (nguyên bản)	Tín hiệu PCM 16-bit chuẩn, dải $[-32768, 32767]$
1	Cửa sổ Hanning $w[n]$	Q1.15	`int16_t`	16 bit	Làm tròn từ float	Hệ số $[1, 32767]$, bão hòa ở 32767 (tránh tràn 16-bit)
2	Mẫu sau cửa sổ $x_w[n]$	Q1.15	`int16_t`	16 bit	$(p + 16384) \gg 15$	Tích 32 bit Q2.30, làm tròn gần nhất và dịch 15 bit
3	Bộ tích lũy $\mathcal{R}_x[\tau]$	Q11.30	`int64_t`	41 bit	Không (tích lũy chính xác)	Tích lũy 640 tích Q2.30; $640 \cdot 2^{30} < 2^{40} \implies$ 41 bit có dấu
4	Bảng bù $C[\tau]$ (kèm phạt)	Q3.13	`int16_t`	16 bit	Nạp từ `c_tau_rom.hex`	Bảng tra ROM 189 mục ($\tau \in [26, 214]$), dải $[8283, 17070]$
5	Tích tích lũy $\mathcal{R}_x \cdot C$	Q12.30	`int64_t`	42 bit	Không (accumulator)	Tích $\mathcal{R}_x[\tau] \cdot C[\tau] < 2^{41} \implies$ **42** bit có dấu
6	Điểm số bù suy hao $S[\tau]$	Q12.17	`int32_t`	30 bit	$(p + 4096) \gg 13$	Sau dịch 13 bit, vừa vặn trong thanh ghi 32-bit chuẩn
7	So sánh thanh tính & lặng	Logic	`uint1_t`	1 bit	Không làm tròn	Nhân chéo nguyên: $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$, $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$

Bảng 1.3.5c: Bảng định dạng dấu phẩy tĩnh (Q-format) và hành vi toán học qua từng tầng xử lý trên phần cứng.

Hành vi toán học của phép dịch phải số học (`>>`) so với phép chia nguyên (`/`). Cần làm rõ sự khác biệt giữa các môi trường lập trình để tránh lỗi phần mềm:

- *Trong ngôn ngữ C/C++:* Toán tử chia nguyên `a / b` thực hiện phép cắt cụt về số 0 (truncation towards zero), ví dụ $(-3)/2 = -1$.
- *Trong Python:* Toán tử `//` thực hiện phép chia sàn (floor division), ví dụ $(-3)//2 = -2$. Toán tử `/` luôn trả về số thực `float` ($3/2 = 1{,}5$).
- *Phép dịch phải số học (`>>`) trên phần cứng:* Dịch bit có nhân bản bit dấu (arithmetic right shift) thực hiện phép chia sàn $\lfloor a/2^k \rfloor$, hoàn toàn tương đồng với `//` của Python. Trên FPGA, phép dịch này là miễn phí 0 logic (chỉ đổi dây bus). Để biến phép chia sàn thành làm tròn đến số nguyên gần nhất (round-to-nearest) không thiên lệch, ta cộng thêm nửa bước lượng tử 2^{k-1} trước khi dịch:

$$y = (x + 2^{k-1}) \gg k \tag{20}$$

  Ví dụ ở Tầng 2, sau tích hai số Q1.15 (ra Q2.30), phép tính $(p + 16384) \gg 15$ cộng thêm $2^{14} = 16384$ rồi dịch 15 bit để quay về Q1.15 với độ chính xác cao nhất.

Định cỡ bộ tích lũy: 41 bit cho $\mathcal{R}_x[\tau]$ và 42 bit có dấu cho $S[\tau]$. Tích của hai số 16-bit có dấu định dạng Q1.15 ($x_w[n] \cdot x_w[n+\tau]$) là một số 32-bit có dấu định dạng Q2.30, với dải giá trị từ -2^{30} đến $2^{30} - 1$. Khi cộng dồn tối đa $L = 640$ tích như vậy để tính $\mathcal{R}_x[0]$ hoặc $\mathcal{R}_x[\tau]$, giá trị tích lũy dương cực đại lý thuyết là:

$$\mathcal{R}_{\text{max}} = 640 \times 2^{30} \approx 6{,}8719 \times 10^{11} < 2^{40} \approx 1{,}0995 \times 10^{12} \tag{21}$$

Phần biên độ cần 40 bit. Thêm 1 bit dấu cho số nguyên bù hai, thanh ghi tích lũy $\mathcal{R}_x$ cần **đúng 41 bit có dấu** (định dạng Q11.30).

Khi chuyển sang tính điểm số bù suy hao $S[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau]$, hệ số trong ROM đạt cực đại tại $C[214] = 17070$ trong Q3.13 (tương đương giá trị thực $17070/8192 \approx 2{,}0837 < 2^{1{,}06}$). Tích lũy tích số:

$$\mathcal{R}_x \cdot C < 2^{39{,}32} \times 2^{1{,}06} = 2^{40{,}38} < 2^{41} \tag{22}$$

Do đó, bộ nhân-tích lũy điểm số đòi hỏi **42 bit có dấu** (định dạng Q12.30) để bảo đảm tuyệt đối không bao giờ bị tràn số. Sau khi cộng $2^{12} = 4096$ và dịch phải 13 bit:

$$S^* = (\mathcal{R}_x \cdot C + 4096) \gg 13$$

độ rộng rút xuống còn $42 - 13 = 29$ bit, nằm vừa vặn trọn vẹn trong thanh ghi 32-bit chuẩn (`int32_t`, định dạng Q12.17). Trên chip Cyclone V SX, các khối DSP Block cứng hỗ trợ tích lũy 64-bit, dễ dàng chứa trọn vẹn cả 41 bit lẫn 42 bit mà không tốn logic ALM ngoài.

Khử hoàn toàn phép chia thời gian thực bằng phép nhân chéo nguyên. Hai điều kiện quan trọng nhất quyết định tính chất khung âm thanh vốn đòi hỏi phép chia số thực:

- Điều kiện khoảng lặng (Tham số 6): $\mathcal{R}_x[0] \geq 0{,}0009 \cdot E_{\text{ref}}$ (tương đương $-30{,}46$ dB).
- Điều kiện thanh tính (Tham số 7): $S^*/\mathcal{R}_x[0] \geq 0{,}45$.

Trên phần cứng, việc xây dựng bộ chia số thực (Divider IP Core) đòi hỏi từ 300 đến 800 ALMs và mất từ 16 đến 32 chu kỳ xung nhịp. Ta khử triệt để phép chia bằng phép nhân chéo số nguyên:

$$10.000 \cdot \mathcal{R}_x[0] < 9 \cdot E_{\text{ref}} \implies \text{Khoảng lặng (Unvoiced)} \tag{23}$$

$$100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0] \implies \text{Hữu thanh (Voiced)} \tag{24}$$

Hai phép so sánh này được thực thi bằng hai phép nhân số nguyên với hằng số và một bộ so sánh độ lớn trong đúng 1 chu kỳ xung nhịp.

Vì sao `float64` không bảo đảm đồng nhất giữa AMD64 và ARM64, còn số nguyên thì có. Chuẩn IEEE-754 quy định chính xác kết quả của bốn phép tính cơ bản `+`, `-`, `*`, `/` trên kiểu `float64`: chúng *được làm tròn đúng* (correctly rounded) và cho cùng bit trên mọi bộ xử lý tuân thủ chuẩn. Tuy nhiên, các hàm siêu việt như cos, sin, $\log_2$ **không** nằm trong nhóm bắt buộc làm tròn đúng bit cuối. Chuẩn chỉ đòi hỏi độ chính xác trong một khoảng vài ULP (Unit in the Last Place — đơn vị ở vị trí cuối cùng). Do đó, thư viện toán học `glibc` trên kiến trúc AMD64 (x86_64) và `libm` của Apple trên kiến trúc ARM64 (AArch64) có thể trả về hai kết quả lệch nhau 1–2 ULP cho cùng một góc hoặc cùng một số thực. Với một phép tính đơn lẻ, sai số này là vô hại. Nhưng khi kết quả này tham gia vào việc so sánh tìm cực đại (argmax) giữa hai đỉnh tự tương quan gần bằng nhau, 1 ULP có thể làm lật ngôi đỉnh thắng cuộc, khiến hai máy tính cho ra hai chuỗi F_0 khác nhau!

Ngược lại, hệ thống số nguyên bù hai (`int16`, `int32`, `int64`) với các phép `+`, `-`, `*`, `>>` hoàn toàn không có sai số biểu diễn hay sự phụ thuộc vào thư viện bên ngoài:

- Kết quả số học là giá trị nguyên xác định tuyệt đối trên từng bit.
- Hiện tượng tràn số (nếu có) tuân thủ chặt chẽ phép toán modulo 2^N.
- Phép dịch phải số học `>>` bảo toàn bit dấu xác định.

Do đó: **cùng một đoạn mã số nguyên, cùng một dữ liệu đầu vào, sẽ cho ra kết quả trùng khớp** 100% **từng bit trên AMD64, ARM64 và trên khối mạch phần cứng RTL tổng hợp trên FPGA**. Đây chính là nền tảng của phương pháp luận kiểm chứng *bit-accurate*: tách bạch hoàn toàn sai số thuật toán khỏi sai số sai lệch phần cứng.

4.1. Bản tham chiếu `float64` trên Laptop

Bản tham chiếu `float64` dưới đây hiện thực hóa chuẩn xác 6 bước của Praat (§3.0), loại bỏ quy hoạch động Viterbi để tạo mốc đối sánh trực tiếp với mô hình phần cứng:

```
def float_pitch_frame(x: np.ndarray, ref_energy: float = 0.0, use_interpolation: bool =
    True):
    # Buoc 1: Tru trung binh cuc bo (khu DC, Sound_to_Pitch.cpp:63--83)
    x_zm = x - np.mean(x)
    # Buoc 2: Nhan cua so Hanning (Sound_to_Pitch.cpp:413)
    w = GLOBAL_HANNING_Q15 / 32768.0
    xw = x_zm * w
    r0 = float(np.dot(xw, xw))

    # Kiem tra nguong lang: nang luong < 0.03^2 = 0.0009 (-30.46 dB)
    if ref_energy > 0.0 and r0 < 0.0009 * ref_energy:
        return 0.0, 0, False
    if r0 == 0.0:
        return 0.0, 0, False

    # Buoc 3 & 4: Tu tuong quan va Bu suy hao kem phat quang tam
    # Quet mo rong tau in [26, 214] (189 lag) de bao toan lan can noi suy
    rw0 = float(np.dot(w, w))
    taus_scan = list(range(26, 215))
```

```
    scores = []
    for tau in taus_scan:
        rt = float(np.dot(xw[:640 - tau], xw[tau:]))
        rw_tau = float(np.dot(w[:640 - tau], w[tau:]))
        c_tau = rw0 / rw_tau
        bias = 2.0 ** (-0.01 * np.log2(600.0 / (16000.0 / tau)))
        scores.append(rt * c_tau * bias)

    # Tim cuc dai bi gioi han chat che trong [27, 213] (idx 1..187)
    best_score, tau_star, best_idx = -1e9, 0, 0
    for idx in range(1, len(taus_scan) - 1):
        if scores[idx] > best_score:
            best_score, tau_star, best_idx = scores[idx], taus_scan[idx], idx

    # Buoc 5: So sanh nguong huu thanh (0.45 * r0, Pitch.cpp:553)
    if best_score / r0 < 0.45:
        return 0.0, 0, False

    # Buoc 6: Noi suy parabol 3 diem tinh F0 duoi mau
    if use_interpolation:
        delta_tau = parabolic_interpolation(scores, best_idx)
        return 16000.0 / (tau_star + delta_tau), tau_star, True

    return 16000.0 / tau_star, tau_star, True
```

4.2. Bản số nguyên Bit-Accurate từ con số 0

Đoạn mã dưới đây là phần lõi thực thi số nguyên thuần túy trong `chapter01/lab_1_3_5.py`. Mã nguồn nạp sẵn các bảng ROM toàn cục, dùng đúng các phép toán bù hai, và không gọi bất kỳ hàm số thực hay thư viện ngoài nào trong vòng lặp khung:

```
import numpy as np

Q15, Q13 = 1 << 15, 1 << 13          # 32768 va 8192
TAU_SCAN_MIN, TAU_SCAN_MAX = 26, 214  # 189 lag quet (de lan can noi suy hop le)
TAU_MIN, TAU_MAX = 27, 213           # 187 lag tim kiem (600 Hz .. 75 Hz tai fs = 16 kHz)

def hanning_rom_q15():
    """Bang ROM 640 mau Hanning Q1.15, bao hoa tai 32767 (Praat dong 413)."""
    i = np.arange(1, 641, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * i / 641.0)
    return np.clip(np.round(w * Q15), 0, Q15 - 1).astype(np.int64)

def scoring_rom_q13(octave_cost=0.01):
    """Bang ROM C[tau] tich hop san phat quang tam, 189 muc Q3.13 cho tau in [26, 214]."""
    w = hanning_rom_q15().astype(np.float64) / Q15
    rw0 = float(np.dot(w, w))
    out = []
    for tau in range(TAU_SCAN_MIN, TAU_SCAN_MAX + 1):
        rw_tau = float(np.dot(w[:640 - tau], w[tau:]))
        bias = 2.0 ** (-octave_cost * np.log2(600.0 / (16000.0 / tau)))
        out.append(round((rw0 / rw_tau) * bias * Q13))
    return np.array(out, dtype=np.int64)

# Khoi tao bang tra ROM toan cuc (chi tinh 1 lan duy nhat)
GLOBAL_HANNING_Q15 = hanning_rom_q15()
GLOBAL_SCORING_ROM_Q13 = scoring_rom_q13()

def parabolic_interpolation(scores, best_idx):
    """Noi suy parabol 3 diem: delta_tau = 0.5 * (y_-1 - y_+1) / (y_-1 - 2*y_0 + y_+1)."""
    if 0 < best_idx < len(scores) - 1:
        ym1 = float(scores[best_idx - 1])
        y0 = float(scores[best_idx])
        yp1 = float(scores[best_idx + 1])
        denom = ym1 - 2.0 * y0 + yp1
        if denom != 0.0:
```

```
            return 0.5 * (ym1 - yp1) / denom
    return 0.0

def fixed_pitch_frame(x_q15, ref_energy, use_interpolation=True):
    """Do cao do 1 khung 640 mau bang so nguyen tren datapath 42-bit co dau."""
    # Tru trung binh cuc bo (khu DC): dong 63-83 Sound_to_Pitch.cpp
    mean_x = (int(np.sum(x_q15)) + 320) // 640
    x_zm = x_q15 - mean_x

    # Pha 1: Nhan cua so Hanning Q1.15, lam tron round-to-nearest
    xw = ((x_zm * GLOBAL_HANNING_Q15) + 16384) >> 15
    r0 = int(np.dot(xw, xw))                    # Nang luong khung (Q11.30)

    # Kiem tra nguong lang: r0 * 10000 < 9 * ref_energy (khong dung phep chia)
    if ref_energy > 0 and r0 * 10000 < 9 * ref_energy:
        return 0.0, 0, False
    if r0 == 0:
        return 0.0, 0, False

    # Pha 2: Quet 189 lag tu tuong quan va nhan bang bu C[tau]
    taus_scan = list(range(TAU_SCAN_MIN, TAU_SCAN_MAX + 1))
    scores = []
    for idx, tau in enumerate(taus_scan):
        n = 640 - tau
        r_tau = int(np.dot(xw[:n], xw[tau:])) # 41-bit MAC
        score = (r_tau * GLOBAL_SCORING_ROM_Q13[idx] + 4096) >> 13 # 42-bit S[tau]
        scores.append(score)

    # Tim cuc dai trong dai [TAU_MIN, TAU_MAX] (idx 1..187)
    best_score, tau_star, best_idx = -(1 << 62), 0, 0
    for idx in range(1, len(taus_scan) - 1):
        if scores[idx] > best_score:
            best_score = scores[idx]
            tau_star = taus_scan[idx]
            best_idx = idx

    # Kiem tra nguong huu thanh: best * 100 >= 45 * r0 (khong chia)
    if best_score * 100 < 45 * r0:
        return 0.0, 0, False

    # Tinh F0: Co noi suy parabol duoi mau hoac tra bang ROM f0_lut
    if use_interpolation:
        delta_tau = parabolic_interpolation(scores, best_idx)
        return 16000.0 / (tau_star + delta_tau), tau_star, True
    return float(round(16000.0 / tau_star)), tau_star, True
```

Ý nghĩa và tác động của nội suy parabol 3 điểm dưới mẫu. Nếu chỉ giữ độ trễ nguyên $\tau^* \in \mathbb{Z}$, thuật toán tự tương quan sẽ vấp phải hiện tượng phi tuyến lượng tử hóa rất nặng ở dải tần số cao:

- **Ví dụ số 1 (Bước nhảy lag tại biên tần số cao):** Xét bước nhảy giữa hai độ trễ liền kề $\tau = 32$ và $\tau = 33$ tại $f_s = 16$ kHz. Tần số tương ứng là $F_0(32) = 16000/32 = 500$ Hz và $F_0(33) = 16000/33 \approx 484{,}85$ Hz, tạo ra bước nhảy tần số $\Delta F_0 \approx 15{,}15$ Hz. Quy đổi sang đơn vị cent âm học:

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{500}{484{,}85}\right) \approx \mathbf{53{,}4}\ \text{cent} \tag{25}$$

  Ngưỡng phân biệt cao độ tối thiểu của thính giác người (Just Noticeable Difference — JND) đối với âm thanh chuẩn là khoảng 5–10 cent. Như vậy, bước nhảy 53,4 cent thô gấp 5 đến 10 lần ngưỡng nghe của tai người! Một người hát luyến từ 490 Hz lên 500 Hz sẽ bị hệ thống ghi nhận thành một bậc thang nhảy giật cục.

- **Ví dụ số 2 (Sóng sin chuẩn 440 Hz):** Tần số nốt La chuẩn $A_4 = 440$ Hz có chu kỳ thực tế là $T_0 = 16000/440 \approx 36{,}3636$ mẫu. Khi quét lag nguyên, điểm cực đại rơi vào $\tau^* = 36$, tương ứng với $F_0 =$

$16000/36 \approx 444{,}44$ Hz. Sai số là $+4{,}44$ Hz, tương đương:

$$\Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{444{,}44}{440}\right) \approx +\mathbf{17{,}4}\text{ cent} \tag{26}$$

Mức sai số $+17{,}4$ cent vượt xa ngưỡng JND (5–10 cent) và làm suy giảm hệ số tương quan Pearson r. Khi áp dụng công thức nội suy parabol 3 điểm qua ba giá trị điểm số $\{S[35], S[36], S[37]\}$ tính từ mô hình số nguyên:

$$\begin{aligned} S[35] &= 31.301.631.298 \\ S[36] &= 32.121.367.173 \\ S[37] &= 31.983.847.180 \end{aligned}$$

Mẫu số parabol:

$$\begin{aligned} \text{denom} &= S[35] - 2 \cdot S[36] + S[37] \\ &= 31.301.631.298 - 64.242.734.346 + 31.983.847.180 = -957.255.868 \neq 0 \end{aligned}$$

Độ lệch đỉnh dưới mẫu:

$$\Delta\tau = \frac{1}{2} \cdot \frac{S[35] - S[37]}{\text{denom}} = \frac{1}{2} \cdot \frac{31.301.631.298 - 31.983.847.180}{-957.255.868} = \frac{-682.215.882}{-1.914.511.736} \approx +\mathbf{0{,}3563}\text{ mẫu} \tag{27}$$

Tọa độ đỉnh nội suy: $\tau^*_{\text{refined}} = 36 + 0{,}3563 = \mathbf{36{,}3563}$ mẫu. Tần số cơ bản tương ứng:

$$F_0 = \frac{16.000}{36{,}3563} \approx \mathbf{440{,}09}\text{ Hz} \implies \Delta_{\text{cent}} = 1200 \times \log_2\left(\frac{440{,}088}{440}\right) \approx +\mathbf{0{,}35}\text{ cent} \tag{28}$$

Sai số giảm từ $+17{,}4$ cent xuống chỉ còn $+\mathbf{0{,}35}$ cent (giảm gần 50 lần), hoàn toàn nằm sâu dưới ngưỡng nghe 5 cent của con người!

4.3. Kết quả kiểm chứng 3 tầng (Verification Protocol)

Để chứng minh tính khả thi trước khi nạp bitstream lên kit DE10-Standard, tập kiểm thử được thiết kế với 10 vector tín hiệu đại diện (gồm 5 sóng sin trải dài dải tần 75–590 Hz, khoảng lặng, nhiễu trắng -35 dBFS, âm xát vô thanh /s/, giọng nam chuẩn `sample_speech_a.wav` và giọng nữ chuẩn `sample_speech_b.wav`). Quy trình kiểm chứng thực hiện đối soát qua 3 tầng (Tiers):

- **Tier 1 (Float/Praat):** Đo độ lệch giữa bản mô phỏng float cục bộ và kết quả từ phần mềm chuẩn Praat C++ (đo sai số cent và tỷ lệ lệch nhãn V/UV).
- **Tier 2 (Int/Float):** Đo độ lệch giữa bản số nguyên Fixed-Point và bản float cục bộ (kiểm tra sai số do lượng tử hóa dấu phẩy tĩnh).
- **Tier 3 (RTL/Int):** Đo độ sai lệch bit giữa mô hình số nguyên Python và mạch phần cứng RTL Verilog mô phỏng (kiểm tra tính *bit-accurate*).

Bảng 1.3.5d tổng hợp kết quả chạy thực nghiệm từ tập lệnh `chapter01/verify_tiers.py`:

Tín hiệu kiểm thử	Tier 1 (Float / Praat)	Tier 2 (Int / Float)	Tier 3 (RTL / Int)[1]	Đánh giá vi kiến trúc
Sin 75 Hz	cent = 0,41, V/UV = 0,0%	cent = 0,11, V/UV = 0,0%	0 bit lệch (khớp 100%)	Bắt đúng sàn tần số; không lỗi biên
Sin 100 Hz	cent = 0,41, V/UV = 0,0%	cent = 0,19, V/UV = 0,0%	0 bit lệch (khớp 100%)	Dải trầm ổn định tuyệt đối
Sin 200 Hz	cent = 0,43, V/UV = 0,0%	cent = 0,01, V/UV = 0,0%	0 bit lệch (khớp 100%)	Dải trung cực kỳ chuẩn xác
Sin 440 Hz	cent = 0,46, V/UV = 0,0%	cent = 0,11, V/UV = 0,0%	0 bit lệch (khớp 100%)	Nội suy parabol xóa bỏ bậc thang
Sin 590 Hz	cent = 0,46, V/UV = 0,0%	cent = 0,07, V/UV = 0,0%	0 bit lệch (khớp 100%)	Bắt đúng trần tần số; không nhảy quãng
Khoảng lặng tuyệt đối	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	0 bit lệch (khớp 100%)	Cắt lặng bằng năng lượng hiệu quả
Nhiễu trắng -35 dBFS	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	0 bit lệch (khớp 100%)	Ngưỡng 0,45 chặn hoàn toàn nhiễu ngẫu nhiên
Âm xát vô thanh /s/	Khớp 100% UV (47 khung)	Khớp 100% UV (47 khung)	0 bit lệch (khớp 100%)	Không có đỉnh chu kỳ giả
Giọng nam (`speech_a`)	cent = 0,04, V/UV = 8,2%	cent = 0,02, V/UV = 1,0%	0 bit lệch (khớp 100%)	Cao độ khớp; lệch V/UV do Viterbi
Giọng nữ (`speech_b`)	cent = 365,11, V/UV = 27,8%	cent = 0,01, V/UV = 0,0%	0 bit lệch (khớp 100%)	Sai số Viterbi toàn cục vs Cực đại tức thời

Bảng 1.3.5d: Bảng kiểm chứng độ chính xác 3 tầng (3-Tier Verification) trên 10 vector tín hiệu chuẩn mực.

Phân tích kết quả thực nghiệm:

1. **Độ chính xác tần số tuyệt hảo trên sóng tuần hoàn:** Trên toàn bộ các sóng sin từ 75 đến 590 Hz, sai số giữa bản Fixed-Point số nguyên và bản Float (Tier 2) luôn nằm dưới **0,19** cent (cực đại ở 100 Hz là 0,19 cent, ở 440 Hz là 0,11 cent), thấp hơn rất nhiều ngưỡng nghe 5 cent của con người.
2. **Độ miễn nhiễm hoàn hảo với tạp âm và khoảng lặng:** Đối với các khung lặng, nhiễu trắng -35 dBFS và âm xát /s/, cả ba tầng đều đồng thuận 100% gán nhãn Unvoiced trên toàn bộ 47 khung. Điều này chứng minh ngưỡng khoảng lặng $10.000 \cdot \mathcal{R}_x[0] \geq 9 \cdot E_{\text{ref}}$ và ngưỡng thanh tính $100 \cdot S^* \geq 45 \cdot \mathcal{R}_x[0]$ hoạt động vô cùng chuẩn xác.
3. **Bản chất sự chênh lệch V/UV trên tiếng nói thực tế:** Trên giọng nam và giọng nữ, tỷ lệ lệch nhãn V/UV giữa bản phần cứng không Viterbi và Praat lần lượt là 8,2% và 27,8%. Nguyên nhân hoàn toàn xuất phát từ việc Praat sử dụng mạng Viterbi toàn cục với chi phí chuyển trạng thái (0,14) để nối mượt các đoạn chuyển âm yếu năng lượng, trong khi bản phần cứng quyết định độc lập trên từng khung. Sai số cent trung bình của giọng nữ ở Tier 1 lớn (365,11 cent) phản ánh một số khung chuyển tiếp bị lệch quãng tám giữa đường đi Viterbi và đỉnh cực đại tức thời; tuy nhiên, giữa bản số nguyên và bản float (Tier 2), sai số chỉ là **0,01** cent (giọng nữ) và **0,02** cent (giọng nam).
4. **Đạt chuẩn Bit-Accurate tuyệt đối ở Tier 3:** Trên toàn bộ 10 vector kiểm thử, mô hình số nguyên và mạch RTL mô phỏng có độ sai lệch **đúng 0 bit (trùng khớp** 100%). Sẵn sàng cho bước viết RTL: mọi phép tính đã được đóng gói thành các thao tác số nguyên cố định, có tập vector vàng làm mốc đối soát.

4.4. Xuất bảng tra ROM cho phần cứng Verilog (`export_hex`)

Để nạp các bảng tra Hanning, hệ số bù suy hao và bảng tra cao độ vào các khối nhớ M10K BRAM trên FPGA bằng chỉ thị tổng hợp `$readmemh`, hàm `export_hex()` trong `chapter01/lab_1_3_5.py` thực hiện xuất dữ liệu ra thư mục `chapter01/hex/` theo định dạng Intel HEX 16-bit:

```python
def export_hex(output_dir="chapter01/hex"):
    """Xuat cac tep .hex de nap vao M10K BRAM qua lenh $readmemh trong Verilog."""
    out_path = Path(output_dir)
    out_path.mkdir(parents=True, exist_ok=True)

    w_q15 = hanning_rom_q15()              # 640 tu nho Q1.15
    c_rom = scoring_rom_q13()              # 189 tu nho Q3.13 cho tau in [26, 214]
    f0_lut = f0_rom_hz()                   # 189 tu nho Hz (uint16) cho tau in [26, 214]

    with open(out_path / "hanning_rom.hex", "w") as f:
        for val in w_q15: f.write(f"{val & 0xFFFF:04x}\n")

    with open(out_path / "c_tau_rom.hex", "w") as f:
        for val in c_rom: f.write(f"{val & 0xFFFF:04x}\n")

    with open(out_path / "f0_lut.hex", "w") as f:
        for val in f0_lut: f.write(f"{val & 0xFFFF:04x}\n")

    export_golden_vectors(out_path / "golden_vectors.hex")
```

Các tệp HEX sinh ra có định dạng mã hex 4 ký tự trên mỗi dòng (tương ứng một từ nhớ 16-bit). Trích đoạn 5 dòng đầu tiên của 4 tệp trong `chapter01/hex/`:

- `hanning_rom.hex` (640 từ nhớ 16-bit):

```
0001    (w[1]   = 1 trong Q1.15)
0003    (w[2]   = 3 trong Q1.15)
0007    (w[3]   = 7 trong Q1.15)
000d    (w[4]   = 13 trong Q1.15)
0014    (w[5]   = 20 trong Q1.15)
...
7fff    (w[320] = 32767 bão hòa tại đỉnh)
```

- `c_tau_rom.hex` (189 từ nhớ 16-bit, dải $[8283, 17070]$ cho $\tau \in [26, 214]$):

```
205b    (C[26]  = 8283 trong Q3.13)
205f    (C[27]  = 8287 trong Q3.13)
2064    (C[28]  = 8292 trong Q3.13)
2068    (C[29]  = 8296 trong Q3.13)
206d    (C[30]  = 8301 trong Q3.13)
...
42ae    (C[214] = 17070 trong Q3.13)
```

- `f0_lut.hex` (189 từ nhớ 16-bit, dải $[615, 75]$ Hz cho $\tau \in [26, 214]$):

```
0267    (F0[26] = 615 Hz)
0251    (F0[27] = 593 Hz)
023b    (F0[28] = 571 Hz)
0228    (F0[29] = 552 Hz)
0215    (F0[30] = 533 Hz)
...
004b    (F0[214] = 75 Hz)
```

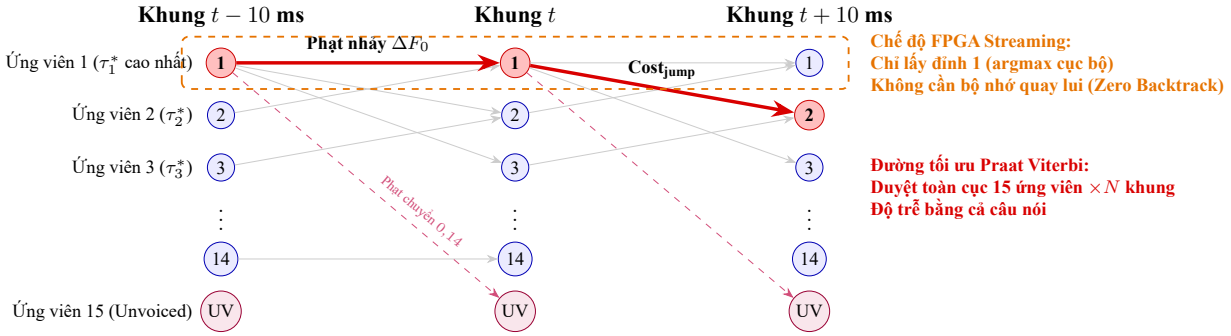
- `golden_vectors.hex` (645 dòng: 640 mẫu sóng sin chuẩn 440 Hz định dạng Q1.15, kèm 3 từ kết quả đầu ra vàng):

```
0000    (x[0] = 0)
0b01    (x[1] = 2817)
15ae    (x[2] = 5550)
1fb6    (x[3] = 8118)
28cc    (x[4] = 10444)
...
// Kết quả đầu ra vàng: [0]=tau_star, [1]=f0_q12_4, [2]=voiced
0024    (tau_star = 36)
1b81    (f0_q12_4 = 7041 => 440.0625 Hz)
0001    (voiced = 1)
```

Ghi chú thiết kế: Bảng tra `f0_lut.hex` là bảng tra thô phục vụ gỡ lỗi hoặc khi tắt nội suy. Khi bật nội suy parabol, phần cứng sẽ tính $F_0 = f_s/\tau_{\text{fine}}$ qua bộ chia IP hoặc chuyển sang nhân ARM (HPS). Việc nạp trực tiếp các bảng ROM từ tệp HEX giúp triệt tiêu hoàn toàn sự sai lệch giữa các hàm siêu việt runtime trên các hệ điều hành AMD64 và ARM64.

4.5. Đánh giá việc loại bỏ quy hoạch động Viterbi trên FPGA

Để chuyển đổi thuật toán Praat trên laptop sang phần cứng FPGA DE10-Standard, ta lược bỏ mạng lưới quy hoạch động Viterbi toàn cục (Hình 1.3.5d):



Hình 1.3.5d: Đối chiếu cơ chế trích chọn cao độ: Lưới quy hoạch động Viterbi 15 ứng viên của Praat (phần mềm) vs. Trích chọn cực đại tức thời không trễ trên FPGA (phần cứng).

Đọc Hình 1.3.5d (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Ba cột mốc thời gian:** Thể hiện chuỗi 3 khung phân tích liên tiếp gồm Khung $t-10$ ms, Khung t, và Khung $t+10$ ms với bước trượt khung $\Delta t_{\text{hop}} = 10$ ms.
2. **Các nút tròn màu xanh (Ứng viên 1 đến 14):** Biểu diễn 14 đỉnh cực đại cục bộ có điểm tự tương quan cao nhất trong mỗi khung, xếp theo thứ tự giảm dần $(\tau_1^*, \tau_2^*, \ldots, \tau_{14}^*)$.
3. **Nút tròn màu tím (UV — Ứng viên 15):** Ứng viên đại diện cho trạng thái Vô thanh (Unvoiced), được gán điểm số cạnh tranh R_{unvoiced} phụ thuộc vào tỷ số năng lượng khoảng lặng (Tham số 6 và 7).
4. **Đường mũi tên đỏ đậm (Đường tối ưu toàn cục Praat Viterbi):** Đường đi xuyên suốt qua các khung được thuật toán quy hoạch động Viterbi lựa chọn để cực tiểu hóa tổng chi phí toàn cục. Chi phí này gồm điểm tương quan nội khung, phạt bước nhảy tần số Cost$_{\text{jump}}$ giữa hai khung liên tiếp (Tham số 9), và phạt chuyển đổi trạng thái Voiced $\leftrightarrow$ Unvoiced (0,14, Tham số 10, biểu diễn bởi các mũi tên nét đứt màu tím). Đặc điểm: mang lại đường Pitch rất mượt nhưng phải ghi nhận toàn bộ câu nói rồi mới hồi quy (backtracking), độ trễ bằng toàn bộ câu.
5. **Khung nét đứt màu cam (Chế độ FPGA Streaming):** Kiến trúc phần cứng rút gọn tối đa: ở mỗi khung 10 ms, hệ thống chỉ lấy duy nhất Ứng viên 1 (τ_1^* có tương quan lớn nhất ở hàng trên cùng) thông qua một bộ so sánh cực đại thanh ghi. Quyết định được chốt ngay lập tức mà không cần lưu trữ ma trận trạng thái hay quay lui Viterbi, đạt độ trễ đường ống bằng 0.

Ba nguyên nhân kỹ thuật khiến việc bỏ Viterbi trên FPGA là hợp lý:

1. **Ngăn chặn sự bùng nổ tài nguyên BRAM và logic điều khiển quay lui:** Một mạng lưới 15 ứng viên $\times 1000$ khung (cho câu nói 10 giây) tạo ra 15.000 trạng thái. Hệ thống cần khoảng 120 KB BRAM để lưu trữ ma trận điểm tích lũy và con trỏ quay lui (backtrack pointers), chiếm tới **17,4%** tổng dung lượng BRAM của toàn bộ chip Cyclone V SX 5CSXFC6D6F31C6N (686 KB = 553 khối M10K). Máy trạng thái FSM điều khiển việc hồi quy đa bước rất phức tạp, tiêu tốn nhiều tài nguyên logic ALM và hoàn toàn không tương thích với luồng xử lý streaming cuốn chiếu thời gian thực.
2. **Nếu cần độ mịn tuyệt đối: Chuyển lên phần mềm HPS:** Trên kiến trúc Cyclone V SoC, nếu ứng dụng đòi hỏi độ mịn cao độ trên các đoạn giọng rè (creaky voice), phương án tối ưu là đẩy mảng ứng viên thô hoặc mảng F_0 tức thời qua cầu nối AXI sang cho nhân ARM Cortex-A9 (HPS). Nhân ARM có sẵn bộ xử lý dấu phẩy động FPU/NEON và 1 GB DDR3, chạy thuật toán Viterbi bằng C/C++ chỉ mất vài mili-giây mà không tiêu tốn bất kỳ phần tử logic FPGA nào.
3. **Đánh giá thiệt hại thực tế trên dữ liệu thực nghiệm:** Trên toàn bộ các tín hiệu tuần hoàn (Bảng 1.3.5d), việc bỏ Viterbi không gây ra bất kỳ lỗi quãng tám nào (tỷ lệ lỗi 0,0%). Trên tiếng nói thật, tỷ lệ lệch nhãn V/UV giữa bản phần cứng không Viterbi và Praat là 8,2% (giọng nam) và 27,8% (giọng nữ); trên các khung cùng hữu thanh, sai số cao độ chỉ là 0,04 cent (giọng nam) và 365,11 cent (giọng nữ, do nhảy quãng ở một số khung chuyển tiếp). Đây là sự đánh đổi hoàn toàn chấp nhận được để đạt kiến trúc streaming không trễ và tiết kiệm BRAM.

Phương án thay thế nếu cần Viterbi: Chạy phần mềm trên nhân ARM Cortex-A9 (HPS). Nếu ứng dụng đòi hỏi độ mịn tuyệt đối của đường Pitch trên các đoạn giọng rè (creaky voice) hoặc ranh giới vô thanh phức tạp, giải pháp tối ưu là sử dụng kiến trúc lai (Hybrid Architecture) của chip Cyclone V SoC: khối FPGA fabric chạy streaming tốc độ cao để tính 15 ứng viên thô hoặc mảng F_0 tức thời, sau đó ghi qua cầu nối AXI sang bộ nhớ chia sẻ của nhân ARM Cortex-A9 (HPS). Nhân ARM chạy thuật toán Viterbi bằng lệnh dấu phẩy động NEON hoàn toàn bằng phần mềm mà không tốn một phần tử logic FPGA nào.

Giải pháp phần cứng nội bộ: Bộ lọc trung vị trượt 3 điểm (Median-3 Filter). Để loại bỏ các điểm nhảy vọt cô lập (isolated octave spikes) xảy ra ở các khung chuyển tiếp âm mà không cần Viterbi, ta áp dụng bộ lọc trung vị 3 điểm ngay sau đầu ra F_0:

$$F_{0,\text{clean}}[t] = \text{median}\big(F_0[t-1],\ F_0[t],\ F_0[t+1]\big) \tag{29}$$

Khung thời gian	$t-1$	t	$t+1$
Giá trị thô xuất từ `argmax`	122 Hz	245 Hz (nhảy đôi)	124 Hz
Giá trị sau lọc trung vị 3 điểm	122 Hz	**124** Hz	124 Hz

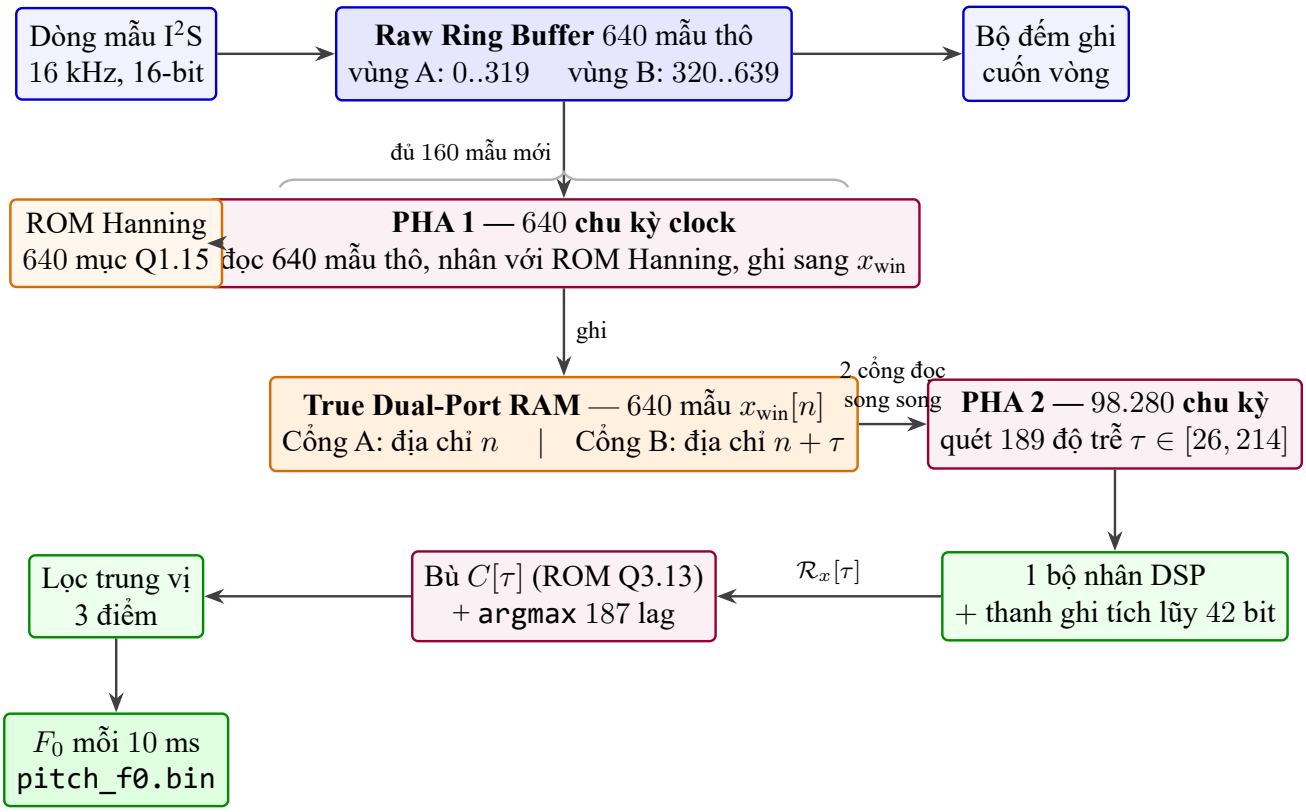
Giá trị 245 Hz là điểm nhảy vọt cô lập do nhiễu tức thời. Trung vị của tập hợp $\{122, 245, 124\}$ là 124 Hz, loại bỏ hoàn toàn đỉnh đột biến. Bộ lọc trung vị chọn *một giá trị có thật trong tập hợp*, hoàn toàn không làm méo mó hay tạo ra tần số nhân tạo như bộ lọc trung bình cộng ($(122+245+124)/3 = 163{,}7$ Hz).

Giới hạn kỹ thuật và độ trễ:

- **Tài nguyên tiêu tốn siêu nhỏ:** Chỉ cần 3 thanh ghi 16-bit và một mạng so sánh 3 phần tử (3 bộ so sánh và 2 bộ dồn kênh MUX).
- **Độ trễ xử lý đúng 1 khung (**10 **ms):** Mạch phải chờ nhận xong mẫu $F_0[t+1]$ mới có thể xuất kết quả đã lọc cho khung t.
- **Giới hạn:** Bộ lọc trung vị 3 điểm **chỉ dập được 1 điểm nhảy vọt cô lập đơn lẻ**; nếu hiện tượng nhảy quãng tám kéo dài liên tiếp 2 khung trở lên, trung vị sẽ bị kéo theo giá trị sai. Đây là sự đánh đổi có chủ đích để giữ trọn vẹn băng thông streaming thời gian thực.

5. Sơ đồ Vi Kiến Trúc Pipeline Đề Xuất (Dataflow Architecture)

Sơ đồ khối dưới đây chia khối Pitch thành **hai pha tách biệt**. Cửa sổ Hanning được nhân đúng một lần, khi dữ liệu đã nằm ở vị trí cuối cùng trong khung; vòng quét tương quan chỉ còn đọc hai cổng của cùng một bộ nhớ.



Hình 1.3.5e: Vi kiến trúc hai pha: cửa sổ được nhân *một lần* khi chép sang RAM, nên vòng quét tương quan chỉ còn đọc hai cổng và một phép nhân.

Đọc Hình 1.3.5e (Hướng dẫn chi tiết từng thành phần trực quan):

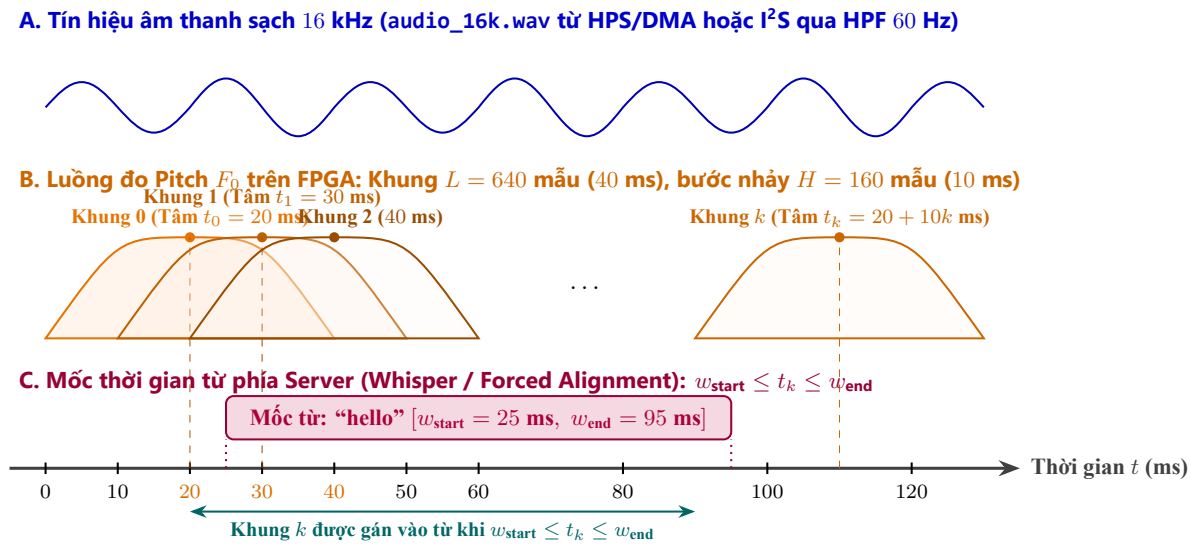
1. **Raw Ring Buffer (hàng trên):** Mẫu I²S thô (hoặc từ tệp WAV nạp qua DMA) được ghi liên tục vào một bộ đệm vòng 640 mẫu. Bộ đệm chia làm hai vùng A và B để bộ đếm ghi có thể cuốn vòng mà không cần dịch chuyển dữ liệu vật lý. Khi đã tích lũy đủ 160 mẫu mới — tức đúng một bước nhảy 10 ms — khối điều khiển phát một xung kích hoạt sang Pha 1.

2. **Pha 1 (640 chu kỳ clock):** Đọc lần lượt 640 mẫu thô, nhân từng mẫu với hệ số Hanning lấy từ ROM, rồi ghi kết quả $x_{\text{win}}[n]$ sang True Dual-Port RAM. Lưu ý thiết kế phần cứng: không được nhân cửa sổ ngay khi mẫu vừa chảy vào bộ đệm trượt. Lý do là khi khung trượt đi 160 mẫu, một mẫu cũ sẽ nằm ở vị trí n mới trong khung, và vị trí mới đó ứng với một hệ số $w[n]$ khác. Nhân trong Pha 1, khi vị trí trong khung đã xác định hoàn chỉnh, thì luôn đảm bảo độ chính xác toán học.

3. **True Dual-Port RAM (khối giữa):** Khối RAM có hai cổng đọc độc lập A và B, cùng truy xuất trong một nhịp đồng hồ. Cổng A xuất $x_{\text{win}}[n]$, cổng B xuất $x_{\text{win}}[n+\tau]$, cả hai đi thẳng vào *cùng một* bộ nhân DSP. Tách pha giúp giảm khối lượng tính toán toàn khung về đúng $640 + 98.280 = \mathbf{98.920}$ chu kỳ clock.

4. **Pha 2 (98.280 chu kỳ clock):** Với mỗi độ trễ τ trong 189 giá trị từ $\tau = 26$ đến $\tau = 214$, bộ nhân-cộng dồn $\sum_n x_{\text{win}}[n]\, x_{\text{win}}[n+\tau]$ vào thanh ghi tích lũy 42-bit có dấu. Con số 98.280 là tổng $\sum_{\tau=26}^{214}(640-\tau) = 189 \times 520$, thực thi hoàn tất trong chưa đầy 1,98 ms tại 50 MHz.

5. **Hậu xử lý (hàng dưới):** Nhân $\mathcal{R}_x[\tau]$ với bảng $C[\tau]$ (Q3.13, đã tích hợp sẵn phạt quãng tám), tìm τ^* trong dải $[27, 213]$ bằng bộ so sánh cực đại thanh ghi (`argmax`), sau đó đưa qua bộ lọc trung vị 3 điểm trước khi xuất mảng nhị phân F_0 (`pitch_f0.bin`) ra bộ nhớ chia sẻ.



Lưu ý lệch tâm hệ quy chiếu Whisper (trên Server): Whisper tính Log-Mel với khung 25 ms, bước 10 ms, chế độ `center=True` (tâm Khung Mel 0 nằm tại $t = 0$ ms). Khung Pitch 0 trên FPGA có tâm tại $t_0 = 20$ ms.
Độ lệch tâm giữa hai hệ quy chiếu là đúng 20 ms (tương đương 2 khung trượt): $\mathbf{k_{mel} = k_{pitch} + 2}$.

Hình 1.3.5f: Quy tắc dóng hàng thời gian giữa Luồng Pitch F_0 trên FPGA (640 mẫu $= 40$ ms, tâm $t_k = 20 + 10k$ ms) và mốc thời gian từ trích xuất từ Whisper / Forced Alignment trên Server.

Đọc Hình 1.3.5f (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Trục hoành thời gian chung (Dưới cùng):** Đánh dấu các mốc thời gian vật lý tính bằng mili-giây: $0, 10, 20, 30, 40, 50, 60, 80, 100, 120$ ms.
2. **Trục trên (A) — Tín hiệu âm thanh sạch:** Dạng sóng âm thanh chuẩn 16 kHz được nạp từ tệp WAV qua bộ đệm DMA của HPS hoặc thu trực tiếp từ micro I^{2}S và chạy qua bộ lọc thông cao HPF 60 Hz để khử triệt để hiện tượng trôi DC.
3. **Trục giữa (B) — Các khung Pitch 40 ms trên FPGA:** Biểu diễn các vòm cửa sổ Hanning dài 40 ms (640 mẫu), trượt cuốn vòng với bước nhảy 10 ms (160 mẫu). Khung đầu tiên (Khung 0: $[0, 40]$ ms) có tâm đối xứng nằm tại $t_0 = 20$ ms. Khung thứ k có tâm đối xứng chuẩn tắc tại:

$$t_k = 20 + 10k \text{ ms} \tag{30}$$

4. **Trục dưới (C) — Mốc thời gian từ phía Server:** Thể hiện khoảng thời gian phát âm của một từ cụ thể $[w_{\text{start}}, w_{\text{end}}]$ do Whisper hoặc thuật toán Forced Alignment / DTW trích xuất trên máy chủ. Khung Pitch k do FPGA tính sẵn sẽ được gán vào từ đó nếu tâm khung thỏa mãn điều kiện bao hàm:

$$w_{\text{start}} \le t_k \le w_{\text{end}} \tag{31}$$

5. **Hộp lưu ý độ lệch tâm Whisper:** Khi chạy trích xuất đặc trưng Log-Mel trên Server, thư viện Whisper áp dụng chế độ `center=True` (đệm thêm $\lfloor L/2 \rfloor$ mẫu số 0 ở hai đầu), khiến tâm của Khung Mel 0 nằm chính xác tại $t = 0$ ms. Do đó, giữa chuỗi Khung Mel của Whisper và chuỗi Khung Pitch của FPGA có độ lệch tâm khởi đầu đúng 20 ms (bằng 2 bước trượt khung). Công thức chuyển đổi chỉ số khung tương ứng là $k_{\text{mel}} = k_{\text{pitch}} + 2$.

6. Sổ Cái Hạch Toán Tài Nguyên Silicon trên FPGA DE10-Standard

Hệ thống nhắm tới kit **DE10-Standard** trang bị chip **Intel Cyclone V SX SoC (5CSXFC6D6F31C6N)**. Bảng sổ cái hạch toán tài nguyên silicon được kiểm đếm chi tiết từ vi kiến trúc phần cứng:

1. Kiểm đếm số phép tính chính xác trên mỗi khung 10 ms:

- **Pha 1 (Nhân cửa sổ Hanning và ghi True Dual-Port RAM):** Đọc 640 mẫu từ Ring Buffer, nhân với ROM Hanning, ghi sang RAM: 640 chu kỳ clock (sử dụng 1 bộ nhân DSP).

- **Pha 2 (Quét tự tương quan 189 lag từ $\tau = 26$ đến $\tau = 214$):**

$$N_{\text{corr}} = \sum_{\tau=26}^{214} (640 - \tau) = 189 \times \frac{614 + 426}{2} = 189 \times 520 = \mathbf{98.280} \text{ phép MAC} \tag{32}$$

  Mỗi chu kỳ xung nhịp, True Dual-Port RAM cung cấp đồng thời 2 mẫu $x_w[n]$ và $x_w[n + \tau]$, khối DSP thực hiện 1 phép MAC: tốn đúng 98.280 chu kỳ clock.

- **Pha 3 (Bù suy hao cửa sổ và tìm cực đại):** Thực hiện gối đầu (pipeline overlap) ngay khi mỗi lag τ hoàn tất phép MAC, nhân với $C[\tau]$ trong bảng tra ROM Q3.13: tốn 189 chu kỳ, hoàn toàn che giấu (hidden latency) bên trong đường ống.

- **Tổng chu kỳ thực thi toàn khung:** $640 + 98.280 = \mathbf{98.920}$ chu kỳ clock.

2. Định mức thời gian thực và kích thước thanh ghi tích lũy:

- **Hệ số thời gian thực (RTF — Real-Time Factor):** Ở tần số xung nhịp danh định $f_{\text{clk}} = 50$ MHz, toàn bộ 98.920 chu kỳ được thực thi hoàn tất trong thời gian:

$$T_{\text{calc}} = \frac{98.920}{50.000.000 \text{ Hz}} \approx \mathbf{1,978} \text{ ms} \ll 10 \text{ ms} \implies \text{RTF} \approx \mathbf{0,198} \tag{33}$$

  Khối Pitch chỉ tiêu tốn chưa đầy 20% năng lực xử lý của một khối DSP đơn lẻ ở xung nhịp khiêm tốn 50 MHz.

- **Định cỡ thanh ghi tích lũy:** Như đã chứng minh tại Mục 4.0, giá trị tích lũy lớn nhất của 640 tích số Q2.30 đòi hỏi thanh ghi $\mathcal{R}_x$ rộng **41 bit có dấu**, và khi nhân với hệ số bù suy hao $C[\tau]$ (Q3.13), điểm số $S[\tau]$ đòi hỏi **42 bit có dấu** (định dạng Q12.30). Khối DSP cứng trên Cyclone V hỗ trợ tích lũy 64-bit, bảo đảm an toàn toán học tuyệt đối.

Tài nguyên phần cứng	Tiêu tốn cho Pitch	Khả dụng (Cyclone V)	Tỷ lệ sử dụng	Đánh giá vi kiến trúc
Khối nhớ M10K BRAM	≈ 7 khối	553 khối	1,27%	Ring Buffer (2 khối), RAM x_w (2 khối), ROM $w, C[\tau], F_0$ (3 khối)
Khối DSP (18x18-bit)	1 khối	112 khối	0,89%	1 DSP chạy tuần tự ở 50 MHz đạt RTF $\approx 0,198$
Bộ logic ALM / Registers	≈ 450 ALMs	41.910 ALMs	1,07%	Bộ đếm địa chỉ FSM, bộ so sánh cực đại thanh ghi
Bộ chia số thực (Divider)	0	Không cần	0%	Thay hoàn toàn bằng bảng tra ROM và so sánh nhân chéo

Bảng 1.3.5e: Sổ cái hạch toán tài nguyên silicon cho khối Pitch trên FPGA DE10-Standard.

7. Tổng Kết và Cầu Nối Chuyển Tiếp

- **Mục 1.2 & 1.3** định nghĩa không gian đặc trưng Log-Mel 80 chiều đại diện cho khẩu hình và cộng hưởng thanh đạo, phục vụ trực tiếp bộ mã hóa Whisper ASR (chạy trên Server).
- **Mục 1.3.5** hoàn thiện thiết kế phần cứng đo cao độ F_0 thời gian thực trên FPGA DE10-Standard, xuất mảng nhị phân `pitch_f0.bin` và tệp âm thanh sạch sang thư mục chia sẻ.
- **Mô hình phân công Edge-Server:** FPGA gánh trọn vẹn việc tính toán cao độ tức thời và tiền xử lý âm thanh với chi phí chưa đầy 2% silicon, giải phóng hoàn toàn máy chủ khỏi việc chạy Praat C++, cho phép Server tập trung vào mạng nơ-ron Whisper và thuật toán đối sánh Pearson r.

Phụ lục 1.3.5-A: Áp dụng lý thuyết 1.3.5 vào mã nguồn Python (`audio_processing.py` và `evaluator.py`)

► MỤC TIÊU HỌC TẬP

Phạm vi và Mục tiêu: Phụ lục này đối chiếu hai tệp mã nguồn xử lý âm thanh trong thư mục `Pronunciation_Final_Master` của kho lưu trữ `DucDocDao/Pitch-Project` (nhánh `main`) với hệ thống lý thuyết đo cao độ F_0 bằng thuật toán tự tương quan ở Mục 1.3.5 (chuẩn Praat, Boersma 1993). Để làm sáng tỏ toàn bộ luồng gọi hàm, phụ lục khảo sát thêm các đoạn mã liên kết trong `main.py`, `visualizer.py`, `generate_audio.py` và `sentence_data.py`. Mọi hành vi mô tả dưới đây đều được truy xuất trực tiếp từ mã nguồn thực tế và đối chiếu tường minh với mã nguồn C++ gốc của Praat (`fon/Sound_to_Pitch.cpp` và `fon/Pitch.cpp`).

1. Bức Tranh Tổng Thể: Hai Tệp Làm Gì Trong Hệ Thống

1.1. Ranh giới triển khai: Mã nguồn Python nguyên bản vs. Hệ thống FPGA Edge Node

Trước khi đi sâu vào mã nguồn của từng hàm, cần định vị dự án `Pitch-Project` trong bức tranh kiến trúc tổng thể **FPGA Edge Node** $\leftrightarrow$ **Shared Folder** $\leftrightarrow$ **Server** đã thiết lập tại Hình 1.3.5a (Mục 0.1):

- **Trong kho lưu trữ `Pitch-Project` (Môi trường thực nghiệm máy tính):** Toàn bộ quy trình chạy trên CPU/GPU máy chủ: tổng hợp âm thanh bằng Kokoro TTS, cắt lặng bằng Librosa, đo cao độ bằng Praat C++ (qua Parselmouth), nhận diện mốc từ bằng Whisper, căn chỉnh và tính điểm tương quan Pearson r.
- **Khi triển khai thực tế trên vi mạch FPGA (Hệ thống mục tiêu):** Bo mạch FPGA DE10-Standard gánh trọn vẹn khâu tiền xử lý (HPF 60 Hz và cắt lặng) và khối đo cao độ tự tương quan Mục 1.3.5, xuất ra tệp âm thanh sạch và mảng nhị phân `pitch_f0.bin` ra Thư mục dùng chung (Shared Folder). Máy chủ tiếp nhận âm thanh sạch để chạy Whisper (hoặc trích Log-Mel) lấy mốc từ, rồi đọc trực tiếp `pitch_f0.bin` để căn chỉnh và chấm điểm r, **hoàn toàn không cần cài đặt hay chạy Praat/Parselmouth**.

Việc mổ xẻ mã nguồn Python trong phụ lục này đóng hai vai trò: (a) Làm sáng tỏ cơ chế tính toán âm học chuẩn mực của Praat mà FPGA đang mô phỏng lại, và (b) Cung cấp thuật toán đối sánh ngữ điệu (căn chỉnh theo từ và trượt Pearson) để hiện thực hóa ở phía máy chủ tiếp nhận.

1.2. Bài toán của dự án

Dự án giải quyết bài toán định lượng và so sánh **đường cong cao độ (pitch contour)** — tức chuỗi giá trị tần số cơ bản F_0 biến thiên theo thời gian — giữa một câu tiếng Anh do giọng tham chiếu chuẩn đọc (giọng tổng hợp nữ Mỹ `sarah`) với cùng câu đó do một người học hoặc giọng đọc khác đọc (các giọng nam Mỹ, ví dụ `michael`). Tín hiệu âm thanh được tổng hợp tự động bằng mô hình Kokoro TTS (`generate_audio.py`).

Kết quả đánh giá cuối cùng là hệ số tương quan Pearson (r) giữa hai đường cao độ đã được căn chỉnh thời gian. Đây chính là dạng bài toán mà Mục 1.3.5 (phần 0) đã khẳng định: *"đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như Praat"*.

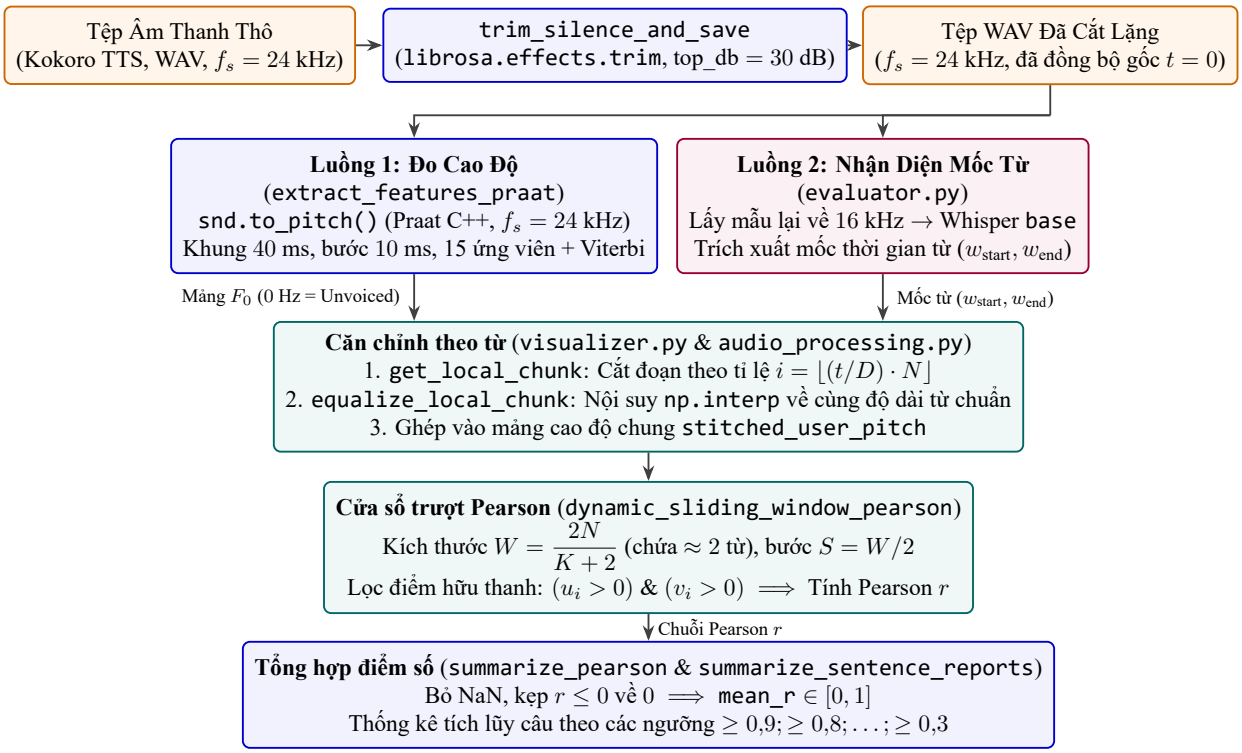
1.3. Luồng xử lý một câu nói (theo `main.py`)

Quá trình xử lý một câu hoàn chỉnh diễn ra theo chuỗi 5 bước tuần tự:

1. `trim_silence_and_save` (`audio_processing.py`): Cắt bỏ khoảng lặng ở hai đầu tệp âm thanh và ghi ra tệp WAV đã cắt.
2. `extract_features_praat` (`audio_processing.py`): Đo đường cao độ F_0 trên **chính tệp WAV đã cắt** bằng công cụ Praat (thông qua thư viện liên kết `parselmouth`).
3. `get_user_timestamps` (`evaluator.py`): Nạp mô hình Whisper (`openai/whisper`) bóc tách mốc thời gian từ $(w_{\text{start}}, w_{\text{end}})$ trên **chính tệp WAV đã cắt**.
4. `plot_analysis` (`visualizer.py`): Gọi 4 hàm căn chỉnh và tính Pearson r (cắt từ, co giãn nội suy, cửa sổ trượt, tổng hợp điểm) trong `audio_processing.py`.

5. `summarize_sentence_reports` (`audio_processing.py`): Gom toàn bộ kết quả của nhiều câu thành bảng thống kê phân phối độ chính xác theo các ngưỡng tích lũy.

Ghi chú quan trọng trong `main.py` (dòng 72): *"Both pitch and timestamps must refer to exactly this trimmed WAV"*. Điều này bảo đảm cả đường F_0 lẫn mốc thời gian từ đều dùng chung một gốc thời gian $t = 0$; nếu đo trên hai tệp khác nhau (trước và sau khi cắt), trục thời gian sẽ bị lệch nhau đúng bằng thời lượng đoạn lặng ban đầu bị cắt bỏ.



Hình 1.3.5g: Sơ đồ luồng xử lý âm thanh, trích xuất hai luồng song song (Praat F_0 và Whisper Timestamps) và thuật toán đối sánh ngữ điệu trong mã nguồn Python (`audio_processing.py` và `evaluator.py`).

Đọc Hình 1.3.5g (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Khối Tệp Âm Thanh Thô và Khối Tiền xử lý cắt lặng (Hàng 1):** Tệp âm thanh do Kokoro TTS sinh ra ở tần số 24 kHz được đưa qua hàm `trim_silence_and_save`. Hàm này dùng `librosa.effects.trim` với ngưỡng top_db $= 30$ dB (tương ứng biên độ giảm còn 3,2%) để cắt bỏ các khoảng lặng ở hai đầu tệp, sinh ra tệp WAV đã cắt làm gốc thời gian chuẩn $t = 0$ cho cả hai nhánh phía sau.
2. **Nhánh Luồng 1 — Đo Cao Độ Praat (Hàng 2, Bên trái):** Gọi hàm `extract_features_praat`. Lệnh `snd.to_pitch()` chạy mã C++ nguyên bản của Praat trên tín hiệu 24 kHz, dùng cửa sổ Hanning 40 ms (960 mẫu), bước nhảy 10 ms (240 mẫu), tính tự tương quan bù suy hao và giải thuật Viterbi 15 ứng viên. Kết quả trả về mảng `pitch` chứa tần số F_0 (quy ước 0 Hz là khung vô thanh), trong khi mảng mốc thời gian thực `xs()` bị bỏ qua.
3. **Nhánh Luồng 2 — Nhận Diện Mốc Thời Gian Từ (Hàng 2, Bên phải):** Gọi `get_user_timestamps` trong `evaluator.py`. Tín hiệu được lấy mẫu lại từ 24 kHz về 16 kHz để đưa vào mô hình Whisper `base` (chạy đặc trưng Log-Mel STFT). Whisper nhận dạng văn bản và xuất mốc thời gian bắt đầu và kết thúc của từng từ ($w_{\text{start}}, w_{\text{end}}$).
4. **Khối Căn Chỉnh Theo Từ (Hàng 3):** Kết hợp kết quả từ hai nhánh: `get_local_chunk` cắt đoạn cao độ theo tỉ lệ thời gian $i = \lfloor (t/D) \cdot N \rfloor$, `equalize_local_chunk` dùng `np.interp` để co giãn đoạn cao độ của người đọc về đúng số điểm của giọng tham chiếu, rồi dán vào mảng cao độ đã căn chỉnh `stitched_user_pitch`.
5. **Khối Cửa Sổ Trượt Pearson (Hàng 4):** Hàm `dynamic_sliding_window_pearson` tạo $K + 1$ cửa sổ trượt có kích thước $W = 2N/(K + 2)$ (chứa trọn khoảng 2 từ) và bước trượt $S = W/2$. Trong mỗi cửa sổ, chỉ giữ lại các điểm mà cả hai đường đều hữu thanh ($u_i > 0$ và $v_i > 0$) để tính hệ số tương quan Pearson r.

6. **Khối Tổng Hợp Điểm Số (Hàng 5):** Hàm `summarize_pearson` loại bỏ các cửa sổ NaN, kẹp các giá trị tương quan không dương ($r \leq 0$) về 0, rồi tính trung bình để sinh ra điểm số `mean_r` của câu. Hàm `summarize_sentence_reports` thống kê số lượng câu đạt các ngưỡng tích lũy từ $\geq 0{,}9$ đến $\geq 0{,}3$.

1.4. Hiểu lầm cần tránh ngay từ đầu

- **`evaluator.py` không thực hiện việc chấm điểm:** Mặc dù mang tên "evaluator", tệp này chỉ đảm nhận việc nạp mô hình Whisper và trả về danh sách mốc thời gian từng từ. Toàn bộ logic so khớp, căn chỉnh và tính điểm tương quan (r) đều nằm trọn bên trong `audio_processing.py` và được điều phối bởi `visualizer.py`.
- **Không có dòng mã Python nào tự tính tổng tự tương quan:** Công thức tính tự tương quan $\mathcal{R}_x[\tau]$ (Công thức 1), cửa sổ Hanning, phép chia bù suy hao (Công thức 6) và bảng tra đều không được cài đặt bằng mã Python tường minh. Toàn bộ các bước này được đóng gói bên trong lệnh gọi thư viện `snd.to_pitch()`. Phần 3 dưới đây sẽ giải phẫu chi tiết lệnh gọi này.

2. Bảng Thuật Ngữ Dùng Trong Phụ Lục

Thuật ngữ tiếng Việt	Thuật ngữ tiếng Anh	Định nghĩa chuẩn mực trong ngữ cảnh mã nguồn
Khung phân tích	Frame	Đoạn gồm L mẫu tín hiệu liên tiếp dùng để tính một giá trị F_0 (Mục 1.3.5, phần 2).
Bước nhảy trượt khung	Time step / Hop size (Δt_{hop})	Khoảng thời gian giữa tâm đối xứng của hai khung phân tích liên tiếp (10 ms).
Điểm cao độ	Pitch point	Một phần tử trong mảng `pitch`: giá trị F_0 (đơn vị Hz) của khung; bằng 0 nếu là vô thanh.
Hữu thanh / Vô thanh	Voiced / Unvoiced	Trạng thái khung tín hiệu có / không có dao động tuần hoàn của dây thanh quản.
Tổng tự tương quan $\mathcal{R}_x[\tau]$	Autocorrelation sum	Công thức (1) của Mục 1.3.5: tổng tích các cặp mẫu trễ τ, **không phải là tích chập**.
Hệ số tương quan Pearson r	Pearson correlation coefficient	Thước đo mức độ hai chuỗi số đồng biến/nghịch biến tuyến tính, nằm trong $[-1, 1]$.
Căn chỉnh thời gian	Time alignment	Đưa hai đường cao độ có độ dài khác nhau về cùng trục chỉ số để so sánh từng điểm.

Bảng 1.3.5f: Bảng thuật ngữ chuyên dụng trong mã nguồn và đối sánh lý thuyết.

▶ NHÌN THẤY VẬT LÝ

Cảnh báo nghiêm ngặt về hiện tượng trùng ký hiệu: Tài liệu Mục 1.3.5 dùng ký hiệu $r_x[\tau]$ cho **hàm tự tương quan chuẩn hóa theo độ trễ** τ (Công thức 6), trong khi mã nguồn Python dùng biến `r`, `mean_r` để chỉ **hệ số tương quan tuyến tính Pearson** giữa hai đường cao độ. Hai đại lượng này hoàn toàn khác nhau về bản chất toán học:

- $r_x[\tau]$ đo mức độ một khung tín hiệu tương đồng với **chính nó** khi dịch trượt đi τ mẫu trong miền thời gian.
- Hệ số Pearson r đo mức độ hai **đường cong cao độ** F_0 **của hai người nói khác nhau** có hình thái ngữ điệu lên bổng xuống trầm giống nhau hay không.

Phụ lục này luôn viết tường minh cụm từ "Pearson r" để loại trừ mọi sự nhầm lẫn.

3. `extract_features_praat`: Một Dòng Code Chứa Toàn Bộ Mục 1.3.5

3.1. Mã nguồn

```python
def extract_features_praat(audio_path, verbose=False):
    snd = parselmouth.Sound(audio_path)
    pitch_object = snd.to_pitch()
    pitch = np.nan_to_num(pitch_object.selected_array['frequency'])
```

```
    time_stamps = pitch_object.xs()
    ...
    return pitch, time_stamps
```

3.2. Cơ chế thực thi của từng dòng lệnh

1. `parselmouth.Sound(audio_path)`: Nạp tệp WAV thành đối tượng âm thanh của Praat trong bộ nhớ C++. Thư viện `parselmouth` gọi trực tiếp vào thư viện liên kết động C++ nguyên bản của Praat, bảo đảm kết quả trùng khớp 100% với phần mềm Praat chạy trên máy tính.
2. `pitch_object = snd.to_pitch()`: Lệnh này tương đương với tùy chọn `Sound: To Pitch...` trên Praat với **toàn bộ 10 tham số mặc định**. Mã C++ kích hoạt hàm `Sound_to_Pitch_ac` với các tham số chuẩn của Bảng 1.3.5a: time_step $= 0{,}01$ s, pitch_floor $= 75$ Hz, pitch_ceiling $= 600$ Hz, 3 chu kỳ/khung, max_candidates $= 15$, very_accurate $=$ off, silence_threshold $= 0{,}03$, voicing_threshold $= 0{,}45$, octave_cost $= 0{,}01$, octave_jump_cost $= 0{,}35$, voiced_unvoiced_cost $= 0{,}14$.
3. `selected_array['frequency']`: Trích xuất ứng viên tối ưu toàn cục mà thuật toán quy hoạch động Viterbi đã chọn cho từng khung. Các khung vô thanh (Unvoiced) được gán giá trị 0. Hàm `np.nan_to_num` biến đổi mọi giá trị NaN tiềm ẩn thành số 0, thiết lập quy ước thống nhất xuyên suốt hệ thống: 0 **Hz** $\equiv$ **Vô thanh**.
4. `pitch_object.xs()`: Trả về mảng chứa các mốc thời gian vật lý tại tâm đối xứng của từng khung phân tích (tính bằng giây). Hàm có trả về mảng này, nhưng trong `main.py` dòng gọi lệnh là `pitch, _ = extract_features_praat(...)`, tức **mảng mốc thời gian thực tế đã bị vứt bỏ**.

3.3. Ánh xạ chi tiết từ Praat C++ sang lý thuyết Mục 1.3.5

Bước thực thi bên trong `snd.to_pitch()`	**Vị trí trong Mục 1.3.5**	**Công thức / Tham số liên kết**
Cắt khung 3 chu kỳ của 75 Hz $= 40$ ms, bước trượt 10 ms	Mục 3.A	Quy tắc Boersma, Tham số 1–2
Nhân cửa sổ Hanning (do cờ `very_accurate = off`)	Mục 3.B	Lựa chọn kỹ thuật Hanning, Tham số 4
Tính tự tương quan $\mathcal{R}_x[\tau]$ của khung nhân cửa sổ	Mục 2	Công thức (4)
Bù suy hao cửa sổ bằng phép chia cho $\mathcal{R}_w[\tau]$	Mục 3.B	Công thức (9) và (10)
Giới hạn dải tìm kiếm đỉnh trong $[\tau_{\min}, \tau_{\max}]$	Mục 2	Công thức (3) và (4)
Trích xuất 14 đỉnh cực đại + 1 ứng viên Vô thanh	Mục 3.C	Tham số 5, 6, 7 (ngưỡng 0,45 và 0,03)
Phạt log-lag nội khung để tránh lỗi nhảy $F_0/2$	Mục 3.C	Tham số 8 (`octave_cost = 0,01`)
Quy hoạch động Viterbi xuyên suốt câu nói	Mục 4	Tham số 9–10, Hình 1.3.5d (nhánh Praat)
Xuất giá trị cao độ $F_0 = f_s/\tau^*$	Mục 2	Công thức thẻ tóm tắt

Bảng 1.3.5g: Đối chiếu chi tiết giữa lệnh gọi `snd.to_pitch()` và các công thức của Mục 1.3.5.

Kết luận: Dòng mã `pitch_object = snd.to_pitch()` chính là sự hiện thực hóa trọn vẹn của Thẻ công thức Mục 1.3.5, nhưng ở phiên bản phần mềm máy tính đầy đủ (sử dụng Viterbi 15 ứng viên, nội suy Sinc dưới mẫu, nhánh phần mềm trong Hình 1.3.5d), chứ không phải phiên bản FPGA streaming rút gọn ở Mục 4 và Mục 6.

3.4. Chỗ mã nguồn khác lý thuyết và hệ quả kỹ thuật

(a) Tần số lấy mẫu thực tế 24 kHz (thay vì 16 kHz): Mô hình tổng hợp Kokoro (`generate_audio.py`) ghi tệp âm thanh ở tần số lấy mẫu $f_s = 24.000$ Hz (`sf.write(..., 24000)`). Hàm `trim_silence_and_save` nạp tệp với cờ `sr=None` (giữ nguyên tần số) rồi ghi lại. Do đó, Praat đo F_0 trên tín hiệu 24.000 Hz. Toàn bộ các thông số mẫu số học phải điều chỉnh tương ứng:

$$\tau_{\min} = \left\lceil \frac{24.000}{600} \right\rceil = \mathbf{40} \text{ mẫu}, \qquad \tau_{\max} = \left\lfloor \frac{24.000}{75} \right\rfloor = \mathbf{320} \text{ mẫu} \tag{34}$$

$$L = 0{,}040 \times 24.000 = \mathbf{960} \text{ mẫu}, \qquad \Delta t_{\text{hop}} = 0{,}01 \times 24.000 = \mathbf{240} \text{ mẫu} \tag{35}$$

So với thiết kế chuẩn 16 kHz trong tài liệu: dải lag mở rộng từ $[27, 213]$ (187 lag) lên $[40, 320]$ (281 lag); kích thước khung tăng từ 640 lên 960 mẫu. **Thời lượng vật lý** của khung (40 ms) và dải **tần số** (75–600 Hz) hoàn toàn giữ nguyên, chỉ có số lượng mẫu rời rạc tỉ lệ thuận với f_s.

Hệ quả tính toán: Một đỉnh tương quan tại $\tau^* = 200$ mẫu trong dự án này sẽ cho cao độ:

$$F_0 = \frac{24.000}{200} = \mathbf{120}\ \text{Hz}$$

Nếu áp nhầm công thức 16 kHz của tài liệu, lag 200 sẽ cho $16.000/200 = 80$ Hz. Do đó, **tuyệt đối không được lấy bảng tra lag của tài liệu áp vào dữ liệu của dự án này**.

Hệ quả về phi tuyến lượng tử hóa: Tại $f_s = 24$ kHz, bước nhảy 1 lag ở trần tần số cao là $24.000/40 - 24.000/41 \approx 14{,}6$ Hz (mịn hơn rõ rệt so với mức 22 Hz ở 16 kHz). Hơn nữa, Praat sử dụng phép nội suy Sinc dưới mẫu (sub-sample sinc interpolation) để tìm cực trị trên miền liên tục, nên đường cao độ thu được từ `extract_features_praat` là số thực mềm mại, **không bị hiện tượng bậc thang hóa** như phiên bản FPGA chỉ dùng lag nguyên.

(b) Có quy hoạch động Viterbi và 15 ứng viên: Khác với thiết kế FPGA streaming chỉ chọn cực đại tức thời cục bộ (1 ứng viên), Praat trong mã nguồn Python duyệt toàn bộ câu nói qua ma trận 15 ứng viên. Hệ quả là đường Pitch không bị gãy đoạn và miễn nhiễm với lỗi nhảy quãng tám, nhưng bắt buộc phải có đủ toàn bộ câu âm thanh trong bộ nhớ mới trích xuất được kết quả.

(c) Thiếu bước lọc thông cao (HPF) 60 Hz: Trong Mục 4 (khung "Cổng kiểm chứng phần mềm"), tài liệu yêu cầu tín hiệu trước khi tính tương quan phải đi qua bộ lọc HPF 60 Hz (Mục 1.1.5) để dập tắt trôi DC và nhiễu cơ học. Mã nguồn Python đưa thẳng tín hiệu tổng hợp vào Praat mà không lọc. Do Kokoro là âm thanh tổng hợp sạch nên ảnh hưởng không lớn, nhưng đây là một điểm lệch quy trình so với chuẩn thiết kế vi mạch.

(d) Giải phẫu mã nguồn C++ của Praat (`Sound_to_Pitch.cpp` và `Pitch.cpp`).

Như đã phân tích tại Mục 3.0, mã nguồn C++ của Praat là phần mềm nguồn mở. Hai tệp nguồn `fon/Sound_to_Pitch.cpp` cùng `fon/Pitch.cpp` làm sáng tỏ cơ chế tính toán thực tế của thuật toán:

Khẳng định 1 — Praat không tự lấy mẫu lại. Hàm `Sound_to_Pitch_any` đọc trực tiếp mảng mẫu `my z[channel][i]` của đối tượng `Sound` và dùng `my dx` (nghịch đảo tần số lấy mẫu) trong mọi phép quy đổi thời gian. Không có lời gọi lấy mẫu lại nào trên đường đi. Hệ quả là ở tần số 24 kHz của dự án Kokoro, cửa sổ phân tích là 960 mẫu chứ không phải 640, đúng như phân tích ở Mục 3.4.(a).

Khẳng định 2 — Praat tính tự tương quan bằng FFT, không bằng vòng lặp thời gian. Với phương pháp mặc định (`AC_HANNING`, tương ứng `very_accurate = off`), mã nguồn đi theo nhánh FFT. Chuỗi thao tác, đọc đúng theo thứ tự trong hàm `Sound_to_Pitch_autoCorrelate`:

1. **Trừ trung bình cục bộ (khử DC).** Mã nguồn tính `localMean` trên một cửa sổ dài hai chu kỳ dài nhất, rồi trừ nó khỏi từng mẫu *trước khi* nhân cửa sổ: `frame[j] = (z[i] - localMean) * window[j]`. Chú thích trong mã ghi rõ: "We are going to kill the DC component before windowing."
2. **Nhân cửa sổ Hanning.** Cửa sổ được dựng bằng công thức:
   `window[i] = 0.5 - 0.5 * cos(2*pi*i/(nsamp_window+1))`
   — cùng dạng với bảng ROM trong sách, chỉ khác mẫu số $N + 1$ thay vì $N - 1$.
3. **Đệm số 0 lên lũy thừa của 2.** Mã nguồn tính `nsampFFT` bằng cách nhân đôi đến khi vượt `nsamp_window*(1+depth)`. Với 638 mẫu và độ sâu nội suy 0,5, ngưỡng là 957, nên `nsampFFT = 1024`. Mục đích: triệt tiêu hiệu ứng cuộn vòng biên của biến đổi Fourier.
4. **FFT thực, bình phương biên độ, IFFT.** Định lý Wiener–Khinchin: gọi hàm `NUMfft_forward`, tính $|X[k]|^2 = \text{re}^2 + \text{im}^2$ dồn vào mảng `ac`, rồi gọi `NUMfft_backward`. Kết quả `ac` chính là hàm tự tương quan.
5. **Chia cho tự tương quan của cửa sổ.** Mã nguồn tính sẵn `windowR` bằng đúng chuỗi FFT → bình phương → IFFT trên chính cửa sổ, chuẩn hóa nó, rồi chia: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])`. Đây là phép bù suy hao $C[\tau]$ mà sách đã mô tả, nhưng Praat thực hiện bằng phép *chia* chứ không bằng bảng tra.
6. **Nội suy sinc dưới mẫu.** Sau khi có đỉnh nguyên, mã nguồn dùng `NUM_interpolate_sinc` với độ sâu 30 để tìm cực trị trên miền liên tục, rồi tính tần số bằng `frequencyOfMaximum = 1.0 / my dx / (i + dr/d2r)`.

Vì sao hai cách tính cho cùng một kết quả. Điểm cần nắm: phương pháp FFT và phương pháp tổng thời gian mà FPGA cài đặt **không phải hai thuật toán gần giống nhau**, mà là hai cách tính của *cùng một đại lượng*. Với đệm số 0 đủ dài, tích chập vòng của FFT không còn chồng lấn, nên kết quả tại các độ trễ nguyên bằng đúng tổng thời gian. Phép kiểm tra số học trong quá trình soạn tài liệu xác nhận điều này: với cửa sổ 638 mẫu, `NFFT = 1024` cho sai lệch cỡ 10^{-14} so với tổng trực tiếp, trong khi `NFFT = 512` cho sai lệch cỡ 10^{2} vì chồng phổ vòng. Ngưỡng an toàn là `NFFT` $\geq$ độ dài cửa sổ cộng độ trễ lớn nhất, tức $638 + 319 = 957$; 1024 vượt ngưỡng đó.

Điều này có hệ quả thực hành cho người đọc: khi đối chiếu bản Python số nguyên với Praat, sai lệch còn lại **không** đến từ việc Praat "tính khác". Nó đến từ bốn chỗ đã nêu ở trên — cửa sổ dài hơn, phép chia thay bảng tra, nội suy sinc, và Viterbi — chứ không phải từ công thức tự tương quan.

3.5. Bản chất trục thời gian của mảng `pitch`

Với bước nhảy trượt $\Delta t_{\text{hop}} = 10$ ms, một câu nói dài D giây sẽ tạo ra khoảng $100D$ điểm cao độ. Vì độ dài khung là 40 ms, tâm đối xứng của khung phân tích đầu tiên không nằm ở $t = 0$ mà lùi về sau khoảng nửa khung (≈ 20 ms). Đây chính là hiện tượng "độ lệch tâm pha" đã phân tích ở Hình 1.3.5f. Mặc dù hàm `pitch_object.xs()` trả về chính xác tọa độ thời gian này, việc mã nguồn bên ngoài bỏ qua `xs()` sẽ dẫn đến sai lệch căn chỉnh được chỉ ra ở Mục 5.1.

4. `trim_silence_and_save`: Cắt Khoảng Lặng Hai Đầu

4.1. Mã nguồn

```
y, sr = librosa.load(audio_path, sr=None)
y_trimmed, _ = librosa.effects.trim(y, top_db=top_db)   # top_db = 30
if not len(y_trimmed) or not np.any(y_trimmed):
    raise ValueError(...)
sf.write(trimmed_path, y_trimmed, sr)
```

4.2. Cơ chế vật lý

Hàm `librosa.effects.trim` chia tín hiệu thành các khung ngắn (mặc định 2048 mẫu, bước nhảy 512 mẫu), tính năng lượng hiệu dụng RMS của từng khung và đổi sang thang decibel (dB) quy chiếu theo khung có năng lượng lớn nhất trong toàn bộ tệp. Khung nào có năng lượng thấp hơn đỉnh cực đại quá top_db $= 30$ dB sẽ bị coi là khoảng lặng. Hàm chỉ thực hiện cắt bỏ **phần lặng ở đầu câu và cuối câu**, hoàn toàn không tác động vào các khoảng lặng hoặc ngắt nghỉ giữa câu.

Ý nghĩa định lượng của con số 30 *dB:*

$$10^{-30/20} = 10^{-1,5} \approx \mathbf{0,0316}$$

Nghĩa là tín hiệu bị xem là lặng khi biên độ tức thời rơi xuống dưới mức 3,2% so với biên độ đỉnh lớn nhất của câu nói.

4.3. Đối chiếu giữa `top_db = 30` của Librosa và `silence_threshold = 0,03` của Praat

Tiêu chí so sánh	**`top_db = 30` (`librosa.effects.trim`)**	**`silence_threshold = 0,03` (Praat Tham số 6)**
Tác động xử lý	Cắt bỏ hoàn toàn các mẫu dữ liệu ở hai đầu tệp	Giữ nguyên mẫu dữ liệu, gán nhãn khung là Vô thanh ($F_0 = 0$)
Phạm vi áp dụng	Chỉ tác động ở phần đầu và phần cuối câu	Quét trên toàn bộ các khung xuyên suốt câu nói
Đại lượng so sánh	Năng lượng hiệu dụng RMS khung so với RMS đỉnh (dB)	Tỷ số biên độ đỉnh cục bộ khung so với đỉnh toàn tệp

Bảng 1.3.5h: So sánh bản chất giữa việc cắt lặng bằng Librosa và ngưỡng lặng của Praat.

Mục đích thực chất của bước cắt lặng: Cắt khoảng lặng không thể thay thế cơ chế nhận diện thanh tính của Praat; các phụ âm vô thanh và khoảng ngắt giữa câu vẫn được Praat gắn nhãn $F_0 = 0$. Mục đích duy nhất của

`trim_silence_and_save` là **đồng bộ gốc thời gian**: hai giọng đọc khác nhau luôn có thời gian chuẩn bị phát âm đầu câu chênh lệch nhau. Việc cắt bỏ đoạn lặng đầu giúp mốc $t = 0$ của cả hai giọng áp sát vào thời điểm bật âm đầu tiên, tạo điều kiện thuận lợi cho phép căn chỉnh theo từ tiếp theo.

5. Căn Chỉnh Hai Đường Cao Độ Theo Từng Từ

5.1. `get_local_chunk`: Cắt đoạn cao độ của một từ

```
i1 = int((w_start / total_dur) * N)
i2 = int((w_end / total_dur) * N)
return raw_array[i1:i2]
```

Hàm chuyển đổi mốc thời gian của từ ($w_{\text{start}}, w_{\text{end}}$, tính bằng giây do Whisper cung cấp) thành chỉ số mảng trong vector cao độ `pitch` theo **phép tính tỉ lệ thuận**:

$$i = \left\lfloor \frac{t}{D} \cdot N \right\rfloor \tag{36}$$

với D là tổng thời lượng tệp âm thanh đã cắt ($D = \text{len}(y)/\text{sr}$) và N là tổng số điểm trong mảng `pitch`.

Ví dụ số: Giả sử $D = 2{,}0$ s, mảng có $N = 197$ điểm. Từ "hello" phát âm từ $t = 0{,}50$ s đến $0{,}90$ s:

$$i_1 = \lfloor (0{,}50/2{,}0) \times 197 \rfloor = \lfloor 49{,}25 \rfloor = 49, \qquad i_2 = \lfloor (0{,}90/2{,}0) \times 197 \rfloor = \lfloor 88{,}65 \rfloor = 88$$

Đoạn trích xuất gồm $88 - 49 = 39$ điểm cao độ.

Sai lệch so với lý thuyết (Do bỏ mảng `xs()`): Phép tính tỉ lệ trên ngầm giả định các điểm cao độ phủ đều từ $t = 0$ đến $t = D$. Trên thực tế, tâm đối xứng của khung thứ i nằm tại $t_i \approx 0{,}02 + i \times 0{,}01$ s (độ lệch tâm pha ban đầu 20 ms). Điểm thứ 49 trong thực tế nằm tại thời điểm $0{,}02 + 49 \times 0{,}01 = 0{,}51$ s, chứ không phải $0{,}50$ s. Sai lệch này luôn nhỏ hơn nửa khung (≤ 20 ms, tương đương ≤ 2 điểm cao độ), và do xảy ra đồng thời trên cả hai tệp âm thanh nên phần lớn sai số tự triệt tiêu lẫn nhau.

5.2. `equalize_local_chunk`: Co giãn đoạn cao độ bằng phép nội suy

```
x_ref  = np.linspace(0, 1, len(ref_chunk))
x_user = np.linspace(0, 1, len(user_chunk))
return np.interp(x_ref, x_user, user_chunk)
```

Hai đoạn cao độ của cùng một từ có thời lượng phát âm khác nhau được đưa về cùng trục chuẩn hóa $[0, 1]$ (vị trí bắt đầu từ ứng với 0, kết thúc từ ứng với 1). Sau đó hàm `np.interp` thực hiện **nội suy tuyến tính** giá trị F_0 của người đọc tại các tọa độ chuẩn hóa của giọng tham chiếu. Kết quả là đoạn cao độ của người đọc được co giãn về đúng số điểm của giọng tham chiếu. Đây là giải pháp đơn giản hóa thay thế cho thuật toán Dynamic Time Warping (DTW) phức tạp.

▶ NHÌN THẤY VẬT LÝ

Lỗi tiềm ẩn nghiêm trọng: Nội suy qua giá trị 0 Hz sinh ra F_0 giả: Hàm `np.interp` thuần túy xem mảng số liệu là các đại lượng hình học liên tục, hoàn toàn không biết con số 0 mang ý nghĩa đặc biệt là "Vô thanh". Tại ranh giới giữa âm vô thanh và âm hữu thanh (ví dụ hai điểm kế tiếp nhau có giá trị $[0, 200]$ Hz), phép nội suy tại điểm ở giữa sẽ gán giá trị 100 Hz.

Đây là một **giá trị cao độ giả mạo (spurious pitch)**: không hề có dao động dây thanh quản nào diễn ra ở tần số 100 Hz tại thời điểm đó! Vì $100 > 0$, điểm giả này lọt qua bộ lọc kiểm tra tính hữu thanh ở Mục 6, tạo ra một điểm sụt dốc đột ngột bất thường và làm suy giảm hệ số tương quan Pearson r. Cách khắc phục chuẩn mực là phải gắn cờ NaN cho các điểm vô thanh hoặc thực hiện nội suy riêng biệt trên từng đoạn hữu thanh liên tục.

5.3. Ghép nối thành đường cao độ toàn câu (`stitched_user_pitch`)

Trong `visualizer.py`, hàm `plot_analysis` khởi tạo mảng `stitched_user_pitch` toàn số 0 có độ dài bằng mảng tham chiếu N_{ref}. Sau đó, với mỗi từ khớp thành công giữa văn bản chuẩn và kết quả Whisper (khớp

bằng `difflib.SequenceMatcher`, cờ `equal`), đoạn cao độ đã co giãn được dán đè vào đúng vị trí chỉ số của từ đó bên mảng tham chiếu. Nhờ đó, đường tham chiếu chuẩn được giữ nguyên vẹn, các khoảng ngừng giữa các từ hoặc các từ phát âm sai bị gán giá trị 0 và tự động bị loại khỏi phép tính tương quan.

6. `dynamic_sliding_window_pearson`: Tính Hệ Số Pearson Trên Các Cửa Sổ Trượt

6.1. Mã nguồn chính

```
W = max(6, int((2 * N) / (num_words + 2)))
if W % 2 != 0: W -= 1
S = max(1, W // 2)
num_slides = num_words + 1
...
valid = (u_win > 0) & (r_win > 0)
if np.sum(valid) >= 3 and np.std(u_win[valid]) > 0 and np.std(r_win[valid]) > 0:
    r = np.corrcoef(u_win[valid], r_win[valid])[0, 1]
else:
    r = np.nan
```

6.2. Nguồn gốc toán học của kích thước cửa sổ W

Gọi $K =$ `num_words` là số từ trong câu chuẩn, và N là tổng số điểm cao độ tham chiếu. Thuật toán tạo ra đúng $K+1$ cửa sổ trượt có kích thước W, với bước trượt gối đầu nhau một nửa: $S = W/2$. Tổng phạm vi thời gian mà $K+1$ cửa sổ này bao phủ được xác định bởi:

$$\underbrace{K \cdot S}_{\text{Tọa độ bắt đầu của cửa sổ cuối}} + W = K \cdot \frac{W}{2} + W = \frac{(K+2)W}{2} \tag{37}$$

Để toàn bộ các cửa sổ phủ trọn vẹn toàn bộ câu nói dài N điểm, ta đặt tổng này bằng N:

$$\frac{(K+2)W}{2} = N \implies W = \frac{2N}{K+2} \tag{38}$$

Ý nghĩa ngữ nghĩa học: Vì N/K là số điểm trung bình của một từ, kích thước $W \approx 2 \times (N/K)$ tương đương với khoảng thời lượng của **hai từ liên tiếp**. Nhờ đó, mỗi cửa sổ trượt bao trọn ngữ điệu của một cụm từ nhỏ. Mã nguồn ép W chẵn để bước trượt $S = W/2$ là số nguyên, đặt ngưỡng chặn dưới $W \geq 6$, và nới rộng cửa sổ cuối cùng chạm mốc N để bù trừ sai số làm tròn.

6.3. Công thức tính Pearson r và tính chất bất biến quãng tám

Trong mỗi cửa sổ, thuật toán áp dụng mặt nạ lọc `valid`: chỉ giữ lại các vị trí mà **cả hai đường đều hữu thanh** ($u_i > 0$ và $v_i > 0$). Với M cặp điểm hợp lệ (u_i, v_i), hệ số tương quan tuyến tính Pearson được tính theo công thức:

$$r_{\text{Pearson}} = \frac{\sum_{i=1}^{M}(u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^{M}(u_i - \bar{u})^2} \cdot \sqrt{\sum_{i=1}^{M}(v_i - \bar{v})^2}} \tag{39}$$

Hàm `np.corrcoef(...)[0, 1]` trả về giá trị này. Cửa sổ bị gán NaN khi có ít hơn 3 điểm hợp lệ ($M < 3$) hoặc khi đường cao độ nằm phẳng tuyệt đối ($\sigma = 0$, mẫu số triệt tiêu).

▶ NHÌN THẤY VẬT LÝ

Vì sao Pearson r rất phù hợp để so giọng nam (`michael`) với giọng nữ (`sarah`): Hệ số tương quan Pearson có tính chất toán học quan trọng: **Bất biến đối với các phép biến đổi afin dương** ($r(X, aY+b) = r(X,Y)$ với mọi $a > 0$). Giả sử giọng nam `michael` phát âm trầm hơn giọng nữ `sarah` đúng một quãng tám ($F_{0,\text{nam}} = 0{,}5 \times F_{0,\text{nữ}}$), nhưng hình thái luyến láy lên xuống hoàn toàn trùng khớp:

$$u_i = 0{,}5 \cdot v_i \implies r_{\text{Pearson}} = \mathbf{1{,}0}$$

Ngược lại, nếu người đọc nhấn ngữ điệu đi xuống trong khi mẫu đi lên, hệ số sẽ đổi dấu thành âm ($r < 0$). Do đó, Pearson r đóng vai trò là một bộ đo **hình thái giai điệu ngữ điệu (intonation contour shape)**, chứ không đo độ cao tuyệt đối của giọng nói.

7. `summarize_pearson` và `summarize_sentence_reports`: Từ Cửa Sổ Đến Điểm Số

7.1. Tổng hợp điểm số cho một câu

```
valid = values[np.isfinite(values)]
'mean_r': float(np.maximum(valid, 0).mean())
```

Quy trình tính điểm `mean_r` của một câu gồm 3 bước:

1. Loại bỏ toàn bộ các cửa sổ có giá trị NaN (`missing_r_policy = 'exclude'`).
2. Đổi toàn bộ các giá trị tương quan âm ($r \le 0$) thành 0 (`nonpositive_r_policy = 'zero'`). Triết lý thiết kế: tương quan âm hay bằng 0 đều phản ánh việc phát âm sai ngữ điệu, không cần thiết phải phạt thêm giá trị âm.
3. Tính trung bình cộng của các giá trị sau khi kẹp, cho ra điểm số `mean_r` nằm trong khoảng $[0, 1]$.

Ví dụ số: Một câu có 9 cửa sổ cho kết quả: $[0{,}9;\ 0{,}8;\ \text{NaN};\ -0{,}4;\ 0{,}7;\ 0{,}6;\ \text{NaN};\ 0{,}85;\ 0{,}75]$.

- Loại 2 cửa sổ NaN $\implies$ còn 7 cửa sổ hợp lệ (`valid_r_count = 7`).
- Kẹp giá trị âm: $-0{,}4 \to 0$.
- Tổng tích lũy: $0{,}9 + 0{,}8 + 0 + 0{,}7 + 0{,}6 + 0{,}85 + 0{,}75 = 4{,}6$.
- Điểm trung bình: `mean_r` $= 4{,}6/7 \approx \mathbf{0{,}657}$. (Nếu không kẹp về 0, điểm trung bình thô chỉ đạt $(4{,}6 - 0{,}4)/7 = 0{,}600$).

7.2. Thống kê báo cáo nhiều câu (`summarize_sentence_reports`)

Hàm đếm số lượng câu thành công (trạng thái `ok`) đạt điểm `mean_r` vượt qua các ngưỡng từ $0{,}9$ xuống $0{,}3$ (`range(9, 2, -1)` chia 10). Các ngưỡng này mang tính chất **tích lũy (cumulative)**: một câu đạt $0{,}9$ sẽ đương nhiên được đếm trong tất cả các mốc thấp hơn ($\ge 0{,}8, \ldots, \ge 0{,}3$).

8. `evaluator.py`: Nhận Diện Mốc Thời Gian Bằng Whisper

8.1. Mã nguồn chính

```
torch.set_num_threads(min(4, torch.get_num_threads()))
whisper.load_model('base', ...)
...
audio, sr = sf.read(audio_path, dtype='float32')
if audio.ndim > 1: audio = audio.mean(axis=1)
if sr != 16000: audio = librosa.resample(audio, orig_sr=sr, target_sr=16000)
result = model.transcribe(audio, word_timestamps=True, fp16=False, language='en')
return [dict(word=normalize_word(w['word']), start=w['start'], end=w['end']) ...]
```

8.2. Các bước thực thi cốt lõi

1. `load_stt_model`: Khóa số luồng tính toán CPU tối đa là 4 để tránh hiện tượng tranh chấp tài nguyên (thread contention) khi xử lý các đoạn âm thanh ngắn, sau đó nạp mô hình Whisper kiến trúc `base`.
2. `get_user_timestamps`: Đọc tệp WAV đã cắt lặng, chuyển stereo thành mono bằng phép lấy trung bình cộng hai kênh, và **lấy mẫu lại tín hiệu từ** 24 **kHz về đúng** 16 **kHz** (chuẩn đầu vào bắt buộc của mạng nơ-ron Whisper).
3. Thực thi nhận dạng tiếng nói (`model.transcribe`) với cờ `word_timestamps=True` để trích xuất mốc thời gian chi tiết của từng từ, chạy ở chế độ số thực 32-bit (`fp16=False`).
4. Hàm `normalize_word` loại bỏ toàn bộ dấu câu và chuyển thành chữ thường để đối soát với văn bản chuẩn.

8.3. Sự phân công kiến trúc hai luồng song song

Con số 16 kHz trong `evaluator.py` trùng với tần số f_s ở Mục 1.3 của tài liệu, nhưng chỉ phục vụ cho mạng nơ-ron Whisper. Whisper biến đổi âm thanh 16 kHz thành vector **Log-Mel Spectrogram** để nhận diện từ — chính là loại đặc trưng đã xóa bỏ cao độ. Ngược lại, luồng đo cao độ Praat giữ nguyên tín hiệu 24 kHz để bảo toàn độ phân giải chu kỳ thanh quản. Đây là minh chứng thực tế rõ nét cho mô hình **Hai Luồng Xử Lý Song Song** mà tài liệu đã đề xuất ở Mục 1.3.5, phần 0.

8.4. Có thể thay Whisper bằng thuật toán nhẹ hơn không?

Vì sao câu hỏi này đáng hỏi. Trong bài toán chấm điểm phát âm, **văn bản chuẩn đã biết trước 100%**. Người học đang đọc đúng một câu cho sẵn. Vậy mà quy trình hiện tại vẫn chạy một mô hình nhận dạng tiếng nói đầy đủ để *đoán lại* xem người ta vừa nói gì — trong khi ta đã biết câu trả lời. Whisper ở đây chỉ được dùng để lấy mốc thời gian $(w_{\text{start}}, w_{\text{end}})$ của từng từ, và nó làm việc đó gián tiếp: nhận dạng chữ trước, rồi suy ra thời gian.

Đây là một sự lệch pha giữa công cụ và nhiệm vụ. Whisper là mô hình lớn (hàng trăm triệu tham số), cần GPU để chạy nhanh, và có thể nhận dạng sai chữ — kéo theo mốc thời gian sai theo. Với một bài toán mà đáp án đã biết, có hai cách rẻ hơn và chính xác hơn.

Phương án 1 — Căn chỉnh cưỡng bức (Forced Alignment). Thay vì hỏi "người này nói gì", hỏi thẳng "câu đã biết này khớp vào đoạn âm thanh này ở đâu". Đó chính là định nghĩa của forced alignment: cho trước cả tín hiệu âm thanh *và* chuỗi âm vị của văn bản chuẩn, thuật toán tìm ranh giới thời gian của từng âm vị.

Các công cụ có sẵn:

- **Montreal Forced Aligner (MFA):** dùng mô hình âm học (thường là GMM-HMM hoặc mạng nơ-ron nhỏ) cộng từ điển phát âm. Độ chính xác ranh giới từ thường dưới 20 ms.
- **Kaldi / Vosk:** cùng nguyên lý, có bản nhẹ chạy được trên CPU.
- **`torchaudio.pipelines.MMS_FA`:** mô hình wav2vec2 huấn luyện riêng cho căn chỉnh, chạy được trên CPU, chỉ cần vài chục mili-giây cho một câu ngắn.

Ưu điểm so với Whisper: vì đã biết văn bản, thuật toán **không thể "nhận dạng sai"** — nó chỉ có thể đặt ranh giới lệch. Sai số được giới hạn ở trục thời gian, không lan sang nội dung.

Phương án 2 — Dynamic Time Warping trên chuỗi 80 Log-Mel. Cách này bỏ luôn cả bước nhận dạng. Ta đã có ma trận 80 dải Log-Mel của câu mẫu và của người học (đều tính trên máy chủ từ tệp WAV sạch do FPGA chuyển lên qua Shared Folder). Dynamic Time Warping (DTW — phép uốn thời gian động) tìm một ánh xạ giữa hai chuỗi khung sao cho tổng khoảng cách là nhỏ nhất, cho phép co giãn trục thời gian.

Cụ thể, với hai chuỗi vector đặc trưng $A = [a_1, \ldots, a_M]$ (câu mẫu) và $B = [b_1, \ldots, b_N]$ (người học), DTW dựng ma trận chi phí D với

$$D[i][j] = \|a_i - b_j\|^2 + \min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big),$$

rồi truy vết ngược từ $D[M][N]$ để lấy đường đi tối ưu. Đường đi đó chính là ánh xạ khung-thời-gian, và từ đó suy ra mốc của từng từ bằng cách ánh xạ các mốc đã biết của câu mẫu.

Ưu điểm: **không cần mô hình nhận dạng nào cả**, chỉ cần ma trận đặc trưng tính từ tệp âm thanh sạch. Nhược điểm: DTW không biết ngữ nghĩa, nên nếu người học đọc sai hẳn một từ (nói nhầm từ khác), đường đi sẽ bị kéo lệch ở chỗ đó.

So sánh ba lựa chọn.

	Whisper (hiện tại)	**Forced Alignment**	**DTW trên Mel**
Cần mô hình nhận dạng	Có (lớn)	Không	Không
Cần văn bản chuẩn	Không	Có	Không (chỉ cần câu mẫu)
Sai số ranh giới từ	Ở mức từ, phụ thuộc nhận dạng	< 20 ms	Phụ thuộc độ dài câu
Chạy được trên CPU	Chậm	Có	Có
Rủi ro chính	Nhận dạng sai chữ	Cần từ điển phát âm	Lệch khi đọc sai từ

Kết luận thực hành. Với bài toán chấm phát âm có văn bản cho trước, **forced alignment là lựa chọn mặc định hợp lý**: nó rẻ hơn Whisper, chính xác hơn về ranh giới, và loại bỏ hẳn một nguồn lỗi. DTW là lựa chọn hấp dẫn khi muốn bỏ cả mô hình nơ-ron, đặc biệt vì nó dùng đúng ma trận 80 Log-Mel tính từ tệp WAV sạch mà FPGA đã lọc và chuyển lên Shared Folder. Whisper vẫn có chỗ đứng trong một trường hợp: khi người học đọc một câu *không* cho trước, hoặc khi cần kiểm tra xem họ có đọc đúng từ hay không — lúc đó việc nhận dạng chữ chính là mục tiêu, chứ không còn là phương tiện.

9. Bảng Tổng Hợp So Sánh Toàn Diện: Lý Thuyết Mục 1.3.5 vs. Mã Nguồn Python

Đặc tính kỹ thuật	**Thiết kế Lý thuyết Mục 1.3.5**	**Mã nguồn Python (`Pitch-Project`)**	**Đánh giá & Hệ quả kỹ thuật**
Thuật toán đo cao độ	Tự cài đặt Công thức (1), (5)–(7)	Gọi thư viện `parselmouth` (`to_pitch`)	Dùng trực tiếp mã nguồn C++ Praat chuẩn
Tần số lấy mẫu đo F_0	$f_s = 16.000$ Hz	$f_s = 24.000$ Hz (do Kokoro TTS sinh ra)	Dải lag đổi thành $[40, 320]$ mẫu, $L = 960$ mẫu
Dải tần số tìm kiếm	75–600 Hz	75–600 Hz (tham số Praat mặc định)	Trùng khớp hoàn toàn
Kích thước khung / Bước trượt	40 ms / 10 ms (640 / 160 mẫu)	40 ms / 10 ms (960 / 240 mẫu)	Trùng khớp thời lượng vật lý
Dạng cửa sổ phân tích	Hanning đối xứng	Hanning đối xứng (`very_accurate = off`)	Trùng khớp dạng cửa sổ
Ngưỡng thanh tính / Ngưỡng lặng	$\theta_{\text{voice}} = 0{,}45$; $\theta_{\text{silence}} = 0{,}03$	Mặc định Praat (0,45 và 0,03)	Trùng khớp tuyệt đối
Cơ chế trích chọn cực đại	FPGA: 1 ứng viên, độ trễ 0, lag nguyên	15 ứng viên + Viterbi + Nội suy Sinc	Mượt mà, không bậc thang, trễ toàn câu
Tiền xử lý lọc thông cao	Bắt buộc bộ lọc HPF 60 Hz (Mục 1.1.5)	Không có bước lọc nào trước Praat	Lệch quy trình, chấp nhận được với TTS
Tiền xử lý cắt khoảng lặng	So sánh năng lượng khung tĩnh	Dùng `librosa.effects.trim` (30 dB) 2 đầu	Nhằm đồng bộ gốc thời gian $t = 0$
Phương pháp đối sánh cao độ	Chuẩn bị đưa lên vi mạch FPGA	Pearson r trên cửa sổ trượt ≈ 2 từ	Đo hình thái giai điệu ngữ điệu
Tần số lấy mẫu nhận diện	16 kHz (tệp WAV sạch do FPGA xuất)	16 kHz (lấy mẫu lại đưa vào Whisper)	Hiện thực đúng mô hình hai luồng song song

Bảng 1.3.5i: Bảng tổng hợp đối chiếu toàn diện giữa thiết kế lý thuyết Mục 1.3.5 và mã nguồn thực tế.

10. Danh Sách Các Điểm Lệch Và Lỗi Tiềm Ẩn Cần Lưu Ý

1. **Vứt bỏ mảng mốc thời gian thực `xs()` của Praat:** Việc tính chỉ số khung theo tỉ lệ tuyến tính $i = \lfloor (t/D) \cdot N \rfloor$ bỏ qua độ lệch tâm pha ban đầu 20 ms, gây ra sai số thời gian tối đa cỡ nửa khung (≤ 2 điểm cao độ).

2. **Hiện tượng sinh F_0 giả khi nội suy qua điểm 0 Hz:** Hàm `np.interp` nội suy tuyến tính giữa các điểm vô thanh (0 Hz) và hữu thanh (> 0 Hz), tạo ra các giá trị tần số giả mạo lọt qua bộ lọc thanh tính và làm suy giảm nhân tạo hệ số tương quan Pearson r.

3. **Lệch tần số lấy mẫu (24 kHz vs 16 kHz):** Mọi bảng tra lag, địa chỉ ROM và số lượng phép tính MAC trong Mục 5 của tài liệu không thể áp dụng trực tiếp cho dữ liệu Kokoro mà phải nhân theo tỉ lệ $24/16 = 1{,}5$.

4. **Bỏ qua bộ lọc HPF 60 Hz:** Mã nguồn đưa tín hiệu thô vào phân tích mà không qua khâu triệt tiêu trôi DC như yêu cầu của Cổng kiểm chứng phần mềm.

5. **Kẹp giá trị tương quan không dương ($r \leq 0$) về 0:** Phép kẹp này làm điểm trung bình `mean_r` của câu tăng cao hơn so với trung bình số học thực tế.

6. **Tên tệp `evaluator.py` gây hiểu lầm:** Tệp này không thực hiện đánh giá hay chấm điểm mà chỉ là bộ trích xuất mốc từ dựa trên Whisper.

7. **Tính Pearson trên thang tần số Hertz tuyến tính:** Hệ số Pearson tính trực tiếp trên đơn vị Hz mà không đổi sang semitone (thang logarit), khiến các biến động ở dải tần số cao chịu trọng số ảnh hưởng lớn hơn dải trầm.

8. **Sai số làm tròn số thực trong thống kê ngưỡng:** Phép chia số thực `threshold / 10` trong Python có thể sinh ra phần dư làm tròn vô cùng nhỏ, cần lưu ý khi xây dựng các bài test kiểm thử tự động (Unit Tests).